University of Austin

Linear Algebra

Mathematics and Computation

Spring 2026

Contents

1 Vector Spaces **2**

2 Subspaces **7**

3 Span and Linear Independence **11**

4 Bases and Dimension **13**

5 Linear Maps **18**

6 Column Space, Null Space, and Rank–Nullity **22**

7 Application: Hamming(7,4) Error-Correcting Code **31**

8 Polynomial Regression **33**

9 Inner Products and Norms (§2.1) **35**

10 Im-Ker Theorem: Full Proof **42**

11 Structural Properties of Linear Transformations (§4) **45**

12 Eigenvalues and Diagonalization (§4.2) **47**

13 Positive Semidefinite Matrices (§4.3) **51**

14 Trace **56**

15 Singular Value Decomposition (§4.4) **56**

16 Determinants (Geometric Approach) **61**

17 Tensor Products **65**

18 Alternating Forms and the Exterior Algebra **68**

Course Philosophy

1. **Math is a productive language**, as opposed to being merely descriptive. A descriptive language names things; a productive language lets you *compute with them, transform them, and derive new facts.* The goal is not to admire structure but to exploit it.

2. **Practical algorithms have two legs to stand on:**

   1. *Practice* — implementation, numerical experiments, real data.
   2. *Theoretical study* — proofs, invariants, guarantees.

   Neither leg alone is stable; the course alternates between them throughout.

Linear algebra studies *vector spaces and their transformations* (up to isomorphism). It is the backbone of modern engineering, ODE/PDE, AI/ML, and scientific computing.

Remark 0.1 (The linearity constraint and why it is enough)**.** Linear algebra is, by definition, limited to *linear* transformations. Most of the world's complexity is *non-linear*. And yet:

- Every smooth non-linear map looks linear at small scales (Taylor / Fréchet derivative).
- Neural networks, though non-linear overall, are built from alternating linear layers and point-wise activations; understanding the linear part is essential.
- Many non-linear problems are solved by repeated linearisation (Newton's method, Gauss–Newton for least squares, Kalman filtering).
- The SVD / PCA gives the best linear summary of any data set—a tool powerful enough to drive modern recommender systems, image compression, and genomics.

So while LA does not capture everything, it captures *exactly the parts we can compute with efficiently.*

1 Vector Spaces

Two questions to keep in mind throughout this section:

- **What is a vector?**
- **What is a vector space?**

These will feel circular at first—and that is intentional.

Back to basics: fields

The scalars come from a *field* $\mathbb{F}$. We primarily use $\mathbb{F} = \mathbb{R}$ and $\mathbb{F} = \mathbb{C}$ (LADR), but the definition works for any field.

Example 1.1 ($\mathbb{Z}_2$: the two-way switch)**.** $\mathbb{Z}_2 = \mathbb{Z}/2\mathbb{Z} = \{0, 1\}$ with arithmetic mod 2:

$$1 + 1 = 0, \quad 0 + 1 = 1, \quad 1 \cdot 1 = 1.$$

Think of $0 =$ off and $1 =$ on. Flipping a switch twice returns to the original state: $1 + 1 = 0$. This is a perfectly valid field, and vector spaces over $\mathbb{Z}_2$ appear in coding theory and cryptography.

Python: arithmetic over $\mathbb{Z}_2$

```python
import numpy as np

# Z_2 arithmetic: addition is XOR, multiplication is AND
def z2_add(a, b): return (a + b) % 2
def z2_mul(a, b): return (a * b) % 2

# Verify field axioms for Z_2 = {0, 1}
elements = [0, 1]
print("Addition table (XOR):")
for a in elements:
    print([z2_add(a, b) for b in elements])

# Vector space over Z_2: binary vectors in Z_2^n
# "addition" = XOR, "scalar mult" = AND with scalar
def z2_vec_add(u, v): return (u + v) % 2
def z2_scalar_mul(lam, v): return (lam * v) % 2

u = np.array([1, 0, 1, 1])
v = np.array([0, 1, 1, 0])
print("u + v =", z2_vec_add(u, v)) # [1,1,0,1]
print("0*u =", z2_scalar_mul(0, u)) # [0,0,0,0]
print("1*u =", z2_scalar_mul(1, u)) # u itself

# Teaser: Hamming(7,4) parity-check matrix over Z_2
# Each column is a non-zero 3-bit binary number
H = np.array([[0,0,0,1,1,1,1],
              [0,1,1,0,0,1,1],
              [1,0,1,0,1,0,1]], dtype=int)

# A valid codeword satisfies Hc = 0 (mod 2)
c = np.array([1,0,1,1,0,1,0]) # a codeword
syndrome = (H @ c) % 2
print("Syndrome (should be 0):", syndrome)

# Flip one bit to introduce an error
c_err = c.copy(); c_err[2] ^= 1
syndrome_err = (H @ c_err) % 2
# The syndrome identifies the column -> the error position
error_col = int("".join(map(str, syndrome_err)), 2)
print("Error at position:", error_col) # 3 (1-indexed)
```

§1.1 Vectors and vector spaces

Definition 1.1. A *vector* is an element of a vector space.

Remark 1.1 (The intentional circularity)**.** The definition above is circular: vectors are defined by the space they live in, and the space is defined by its vectors. This is not a flaw—it reflects that the concept of "vector" has no meaning in isolation. What matters is the *structure* (the operations and axioms), not the nature of the objects themselves. A polynomial, a matrix, and an arrow in $\mathbb{R}^3$ can all be vectors if they satisfy the same axioms.

Definition 1.2. A *vector space* V over a field $\mathbb{F}$ is a set of vectors equipped with two operations:

- addition $+ : V \times V \to V$,
- scalar multiplication $\cdot : \mathbb{F} \times V \to V$,

satisfying the following eight axioms. The quantifiers apply to *all* $u, v, w \in V$ and *all* $\lambda, \mu \in \mathbb{F}$ unless stated otherwise.

1. **Addition associativity:** $\forall\, u, v, w \in V : \quad (u + v) + w = u + (v + w)$.
2. **Addition commutativity:** $\forall\, u, v \in V : \quad u + v = v + u$.
3. **Additive identity:** $\exists\, 0 \in V\ \forall\, v \in V : \quad v + 0 = 0 + v = v$.
4. **Additive inverse:** $\forall\, v \in V\ \exists\, u \in V : \quad v + u = 0$. *(we write $u = -v$)*
5. **Scalar associativity:** $\forall\, \lambda, \mu \in \mathbb{F},\ v \in V : \quad (\lambda\mu)v = \lambda(\mu v)$.
6. **Scalar distributivity:** $\forall\, \lambda, \mu \in \mathbb{F},\ v \in V : \quad (\lambda + \mu)v = \lambda v + \mu v$.
7. **Vector distributivity:** $\forall\, \lambda \in \mathbb{F},\ u, v \in V : \quad \lambda(u + v) = \lambda u + \lambda v$.
8. **Scalar identity:** $\exists\, 1 \in \mathbb{F}\ \forall\, v \in V : \quad 1 \cdot v = v$.

Examples

Remark 1.2 (Examples are important)**.** The axioms are necessary but dry. Examples are where intuition lives. Every time you meet a new vector space, ask: what do addition and scalar multiplication *mean* here? What does the zero vector look like?

Example 1.2.

1. $V = \mathbb{F}^n$, $n \in \mathbb{N}$, $n \geq 1$. This is the *model example*: every finite-dimensional vector space is isomorphic to some $\mathbb{F}^n$, so understanding $\mathbb{F}^n$ thoroughly means understanding all of finite-dimensional linear algebra.
2. $V = \{0\}$ (the trivial vector space, with $0 + 0 = 0$ and $\lambda \cdot 0 = 0$).
3. **Function spaces.** Let X be any set. Then

   $$\mathbb{F}^X = \{f : X \to \mathbb{F}\}$$

   is a vector space under *pointwise* operations: $(f + g)(x) = f(x) + g(x)$ and $(\lambda f)(x) = \lambda f(x)$.

   Key sub-example. Take $X = [0, 1]$ and $\mathbb{F} = \mathbb{R}$. Then $\mathbb{R}^{[0,1]}$ is the space of *all* real-valued functions on $[0, 1]$—it is *infinite-dimensional.* The trigonometric functions $\{1, \cos(n\pi x), \sin(n\pi x)\}_{n\geq 1}$ form an orthogonal basis for the subspace of square-integrable functions (this is the starting point of *Fourier analysis*).
4. **RGB colors.** $c = (r, g, b)^T$ with $r, g, b \in \{0, 1, \ldots, 255\}$.

   Exercise 1.1. Is the set of RGB color vectors a vector space over $\mathbb{R}$? If not, which axiom(s) fail?

5. **Polynomials of degree $\leq n$.**

$$\mathcal{P}_n = \{a_0 + a_1 x + \cdots + a_n x^n : a_i \in \mathbb{R}\}.$$

Addition and scalar multiplication are defined coefficient-wise. The map

$$\varphi : \mathcal{P}_n \longrightarrow \mathbb{R}^{n+1}, \qquad \varphi(a_0 + a_1 x + \cdots + a_n x^n) = (a_0, a_1, \ldots, a_n)^T$$

is a bijection that respects both operations: $\varphi(\lambda p + \mu q) = \lambda\varphi(p) + \mu\varphi(q)$. Such a structure-preserving bijection is called an *isomorphism* — a "sort of equality" between seemingly different objects. So $\mathcal{P}_n \cong \mathbb{R}^{n+1}$ as vector spaces.

Exercise 1.2. Do polynomials of degree *exactly* n (i.e., $a_n \neq 0$) form a vector space? Which axiom fails?

Python: function spaces as vectors — signals

```
import numpy as np
import matplotlib.pyplot as plt

# Discretise [0, 2pi] into N points: each signal is a vector in R^N
N = 512
t = np.linspace(0, 2 * np.pi, N)

# Two "vectors" in the function space R^[0,2pi]
f = np.sin(t) # f(t) = sin(t)
g = np.cos(2 * t) # g(t) = cos(2t)

# Vector addition and scalar multiplication are pointwise
h = f + 0.5 * g # h = f + 0.5*g (a new vector in the same space)

# Verify axiom 3 (zero vector): the zero function
zero = np.zeros(N)
print("f + zero == f:", np.allclose(f + zero, f))

# Verify axiom 4 (additive inverse): -f
print("f + (-f) == 0:", np.allclose(f + (-f), zero))

# Verify axiom 7 (distributivity): lambda*(f+g) == lambda*f + lambda*g
lam = 3.7
print("Distributivity:", np.allclose(lam * (f + g), lam * f + lam * g))

# Plot: three "vectors" in the function space
plt.figure(figsize=(8, 3))
plt.plot(t, f, label=r'$f = \sin t$')
plt.plot(t, g, label=r'$g = \cos 2t$')
plt.plot(t, h, label=r'$h = f + 0.5g$', linestyle='--')
plt.legend(); plt.title("Addition in the function space $\mathbb{R}^{[0,2\pi]}$")
plt.tight_layout(); plt.show()
```

Formal consequences of the axioms

Remark 1.3 (Why formal proofs? Universality.)**.** The propositions below are not hard. The point is not the results themselves—they feel obvious. The point is the *technique*: every step must follow from the axioms alone, with no appeal to a specific example. A proof that uses only the axioms works for *every* vector space simultaneously: $\mathbb{R}^n$, function spaces, polynomial spaces, matrix spaces, spaces over $\mathbb{Z}_2$, spaces you have not yet imagined. This is *universality.* Abstraction is not the goal; universality is.

Proposition 1.1 (1.1.1 — Uniqueness of zero)**.** *A vector space has a unique additive identity* 0*.*

Proof. Suppose 0 and $0'$ both satisfy axiom 3 (additive identity). Then:

$$0 \overset{\text{ax. 3 on 0}}{=} 0 + 0' \overset{\text{ax. 2}}{=} 0' + 0 \overset{\text{ax. 3 on 0}'}{=} 0'. \qquad \square$$

Proposition 1.2 (1.1.2 — Uniqueness of additive inverse)**.** *Every vector* $v \in V$ *has a unique additive inverse.*

Proof. Suppose u and w both satisfy axiom 4 for v: $v + u = 0$ and $v + w = 0$. Then:

$$u \overset{\text{ax. 3}}{=} u + 0 \overset{v+w=0}{=} u + (v + w) \overset{\text{ax. 1}}{=} (u + v) + w \overset{u+v=0}{=} 0 + w \overset{\text{ax. 2, 3}}{=} w. \qquad \square$$

Proposition 1.3 (1.1.3)**.** $\forall\, v \in V\colon\ 0_{\mathbb{F}} \cdot v = 0_V$*.*

Proof. In $\mathbb{F}$, $0_{\mathbb{F}} + 0_{\mathbb{F}} = 0_{\mathbb{F}}$, so:

$$0_{\mathbb{F}} \cdot v = (0_{\mathbb{F}} + 0_{\mathbb{F}}) \cdot v \overset{\text{ax. 6}}{=} 0_{\mathbb{F}} \cdot v + 0_{\mathbb{F}} \cdot v.$$

Adding $-(0_{\mathbb{F}} \cdot v)$ (which exists by axiom 4) to both sides: $0_V = 0_{\mathbb{F}} \cdot v$. $\square$

Proposition 1.4 (1.1.4)**.** $\forall\, \lambda \in \mathbb{F}\colon\ \lambda \cdot 0_V = 0_V$*.*

Proof. By axiom 3, $0_V + 0_V = 0_V$, so:

$$\lambda \cdot 0_V = \lambda \cdot (0_V + 0_V) \overset{\text{ax. 7}}{=} \lambda \cdot 0_V + \lambda \cdot 0_V.$$

Adding $-(\lambda \cdot 0_V)$ to both sides: $0_V = \lambda \cdot 0_V$. $\square$

Proposition 1.5 (1.1.5)**.** $\forall\, v \in V\colon\ (-1) \cdot v = -v$*.*

Proof. Compute $v + (-1) \cdot v$:

$$v + (-1) \cdot v \overset{\text{ax. 8}}{=} 1 \cdot v + (-1) \cdot v \overset{\text{ax. 6}}{=} (1 + (-1)) \cdot v = 0_{\mathbb{F}} \cdot v \overset{\text{Prop. 1.1.3}}{=} 0_V.$$

So $(-1) \cdot v$ is an additive inverse of v. By Prop. 1.1.2 the inverse is unique, hence $(-1) \cdot v = -v$. $\square$

2 Subspaces

Remark 2.1 (General philosophy)**.** The ambient vector-space structure is trivial in the sense that the axioms are simple and few. But within that shell, non-trivial structure emerges: subspaces, spans, bases, linear maps. The game is always the same — start from the universal axioms, derive consequences that hold for *every* vector space at once.

Notation. V denotes a vector space over $\mathbb{F}$ (take $\mathbb{F} = \mathbb{R}$ whenever a specific field is not mentioned).

Definition 2.1 (§1.2)**.** A subset $U \subseteq V$ is a *subspace* (written $U \leq V$) if:

1. $0 \in U$,
2. U is closed under addition: $\forall\, u, v \in U :\ u + v \in U$,
3. U is closed under scalar multiplication: $\forall\, \lambda \in \mathbb{F},\ u \in U :\ \lambda u \in U$.

Proposition 2.1 (Subspace test — one condition)**.** *A nonempty subset $U \subseteq V$ is a subspace if and only if*

$$\forall\, u, v \in U,\ \forall\, \lambda \in \mathbb{F} : \quad \lambda u + v \in U.$$

Proof. ($\Rightarrow$) If U is a subspace, closure under scalar multiplication gives $\lambda u \in U$, and then closure under addition gives $\lambda u + v \in U$.

($\Leftarrow$) Set $\lambda = 0$: $0 \cdot u + v = v \in U$, so U is closed under addition. Set $v = 0$: $\lambda u + 0 = \lambda u \in U$, so U is closed under scalar multiplication. For the zero vector, pick any $u \in U$ (nonempty) and set $\lambda = 0$, $v = 0$: $0 \in U$. □

Example 2.1 (Trivial subspaces)**.** Every vector space V has exactly two *trivial subspaces*: $\{0\}$ and V itself.

- $\{0\}$ satisfies the definition for the dumbest possible reason: it contains only the zero vector, so every condition is vacuously or trivially met.
- V satisfies the definition because it is already a vector space, so closure holds by the axioms.

Any subspace strictly between $\{0\}$ and V is *non-trivial*, and these are the interesting ones.

Example 2.2 (Non-example: affine shifts are not subspaces)**.** Let $U \leq V$ be a subspace and $c \in V$ with $c \neq 0$. The *affine set*

$$U + c = \{u + c : u \in U\}$$

is *not* a subspace: it fails condition 1, since $0 = u + c$ would require $u = -c \in U$, but then $c = -u \in U$ (as U is closed under scalar mult with $\lambda = -1$), contradicting $c \notin U$ if $c \neq 0$ and $0 \notin U + c$ in general.

Concrete instance. In $\mathbb{R}^2$, the line $\ell = \{(x, 1) : x \in \mathbb{R}\}$ is an affine shift of the x-axis (a subspace) by the vector $(0, 1)^T$. It does not pass through the origin, so it is not a subspace.

Example 2.3 ($\mathcal{P}_1 \leq \mathcal{P}_3$)**.** Let $V = \mathcal{P}_3$ and $U = \{mx + b : m, b \in \mathbb{R}\}$ (polynomials of degree ≤ 1). We verify the three conditions:

1. $0 \in U$: take $m = b = 0$, giving the zero polynomial. ✓
2. Closed under addition: $(m_1 x + b_1) + (m_2 x + b_2) = (m_1 + m_2)x + (b_1 + b_2) \in U$. ✓

3. Closed under scalar mult: $\lambda(mx+b) = (\lambda m)x + (\lambda b) \in U$. ✓

Hence $U = \mathcal{P}_1 \leq \mathcal{P}_3$.

Example 2.4 (Homogeneous vs. inhomogeneous linear constraints)**.** Let $V = \mathbb{R}^4$ and $b \in \mathbb{R}$ a fixed constant. Consider

$$U_b = \{(x_1, x_2, x_3, t)^T \in \mathbb{R}^4 : x_3 = 5x_1 + b\}.$$

Claim: U_b is a subspace if and only if $b = 0$.

Proof. ($b = 0$) The constraint $x_3 = 5x_1$ is *homogeneous* (passes through the origin). Check: $0 \in U_0$ since $0 = 5 \cdot 0$. If $u, v \in U_0$ then $(u+v)_3 = u_3 + v_3 = 5u_1 + 5v_1 = 5(u+v)_1$, so $u+v \in U_0$. Similarly $(\lambda u)_3 = \lambda u_3 = 5\lambda u_1$, so $\lambda u \in U_0$. Hence $U_0 \leq V$.

($b \neq 0$) The zero vector requires $0 = 5 \cdot 0 + b = b \neq 0$, a contradiction. So $0 \notin U_b$, and condition 1 fails. □

Moral: a linear constraint defines a subspace iff it is *homogeneous* (constant term zero). Inhomogeneous constraints give affine subsets, not subspaces.

Sum and direct sum of subspaces

Definition 2.2. Let $V_1, V_2, \ldots, V_m \leq V$ be subspaces. Their *sum* is

$$V_1 + V_2 + \cdots + V_m = \{v_1 + v_2 + \cdots + v_m : v_i \in V_i,\ i = 1, \ldots, m\}.$$

Exercise 2.1. What is the sum of

$$U = \{(x, x, y, y)^T \in \mathbb{R}^4 : x, y \in \mathbb{R}\} \quad \text{and} \quad W = \{(x, x, x, y)^T \in \mathbb{R}^4 : x, y \in \mathbb{R}\}?$$

Proposition 2.2 (1.2.1)**.** *Let $V_1, \ldots, V_m \leq V$ be subspaces. Then $V_1 + \cdots + V_m$ is the* smallest *subspace of V containing each V_i.*

Proof. It is a subspace. $0 = 0 + \cdots + 0 \in V_1 + \cdots + V_m$ (taking $v_i = 0 \in V_i$). If $w = \sum v_i$ and $w' = \sum v_i'$ are two elements, then $w + w' = \sum(v_i + v_i')$ with $v_i + v_i' \in V_i$, so $w + w'$ is in the sum. Similarly $\lambda w = \sum(\lambda v_i)$ with $\lambda v_i \in V_i$.

It contains each V_i. Given $v \in V_i$, write $v = 0 + \cdots + 0 + v + 0 + \cdots + 0$ (v in position i, zeros elsewhere); this is in $V_1 + \cdots + V_m$.

It is the smallest. Let $W \leq V$ be any subspace containing each V_i. Any element $\sum v_i$ (with $v_i \in V_i \subseteq W$) belongs to W by closure under addition, so $V_1 + \cdots + V_m \subseteq W$. □

Definition 2.3 (Direct sum)**.** The sum $W = V_1 + \cdots + V_m$ is a *direct sum*, written $W = V_1 \oplus \cdots \oplus V_m$, if every $w \in W$ can be written in *exactly one way* as $w = v_1 + \cdots + v_m$ with $v_i \in V_i$.

Proposition 2.3 (Equivalent characterizations of direct sum)**.** *The following are equivalent:*

(i) $W = V_1 \oplus \cdots \oplus V_m$.

(ii) $W = V_1 + \cdots + V_m$ *and* $v_1 + \cdots + v_m = 0$ *with* $v_i \in V_i$ *implies* $v_i = 0$ *for all* i.

(iii) $W = V_1 + \cdots + V_m$ *and* $V_i \cap \sum_{j \neq i} V_j = \{0\}$ *for all* i.

Special case $m = 2$***:*** $W = V_1 \oplus V_2$ *iff* $W = V_1 + V_2$ *and* $V_1 \cap V_2 = \{0\}$.

Proof. (i)⇒(ii): If the representation is unique and $v_1 + \cdots + v_m = 0$, then $0 + \cdots + 0 = 0$ is another representation, so by uniqueness each $v_i = 0$.

(ii)⇒(iii): Suppose $u \in V_i \cap \sum_{j \neq i} V_j$. Then $u = \sum_{j \neq i} v_j$ for some $v_j \in V_j$, so $v_1 + \cdots + (-u) + \cdots + v_m = 0$ (with $-u$ in position i). By (ii) each term is 0, so $u = 0$.

(iii)⇒(i): If $\sum v_i = \sum v_i'$ then $\sum (v_i - v_i') = 0$, so $v_i - v_i' = -\sum_{j \neq i} (v_j - v_j') \in V_i \cap \sum_{j \neq i} V_j = \{0\}$, giving $v_i = v_i'$. □

Python: direct sum decomposition and uniqueness

```
import numpy as np

# R^4 = V1 + V2 where:
# V1 = span{e1, e2} (first two coords free, last two zero)
# V2 = span{e3, e4} (first two zero, last two coords free)
# This is a direct sum: V1  V2 = {0}.

def project_V1(v): return np.array([v[0], v[1], 0, 0])
def project_V2(v): return np.array([0, 0, v[2], v[3]])

v = np.array([3., -1., 2., 5.])
v1 = project_V1(v)
v2 = project_V2(v)

print("v =", v)
print("v1 =", v1, " (component in V1)")
print("v2 =", v2, " (component in V2)")
print("v1 + v2 == v:", np.allclose(v1 + v2, v)) # True: decomposition works
print("V1  V2 = {0}:", np.allclose(project_V1(v2), 0) and
                       np.allclose(project_V2(v1), 0))

# Now a NON-direct sum: V3 = span{e1+e2}, V4 = span{e1}
# V3 + V4 = span{e1, e2} but V3  V4  {0}
V3_basis = np.array([1., 1.]) # in R^2 for simplicity
V4_basis = np.array([1., 0.])
w = np.array([2., 1.])

# Two different decompositions of w in V3 + V4:
# w = 1*(1,1) + 1*(1,0) -> coeffs (1, 1)
# w = 0*(1,1) + 2*(1,0) -> not valid since 0*(1,1) is in V3 but
# Actually let's solve: w = a*(1,1) + b*(1,0)
# => a+b=2, a=1 => unique. But try a degenerate case:
V3b = np.array([1., 1.])
V4b = np.array([2., 2.]) # parallel to V3b -> NOT direct sum
A = np.column_stack([V3b, V4b])
print("\nParallel subspaces (not direct sum):")
print("Rank of [V3, V4]:", np.linalg.matrix_rank(A),
      " (should be 1, not 2 -> not direct sum)")
```

Python: numerically testing the subspace conditions

```
import numpy as np
```

```python
rng = np.random.default_rng(42)

def is_subspace_numeric(sample_fn, n_trials=2000, tol=1e-10):
    """
    Numerically test whether a set U (described by sample_fn) looks
    like a subspace of R^n. sample_fn() should return a random element of U.
    Tests: (1) zero in U, (2) closed under addition, (3) closed under scaling.
    """
    # Condition 1: zero vector
    zero = np.zeros_like(sample_fn())
    # We check it by seeing if 0*u is in U (i.e. looks like a sample)
    # Here we just verify sample_fn produces things near the zero-pattern

    failures = 0
    for _ in range(n_trials):
        u = sample_fn()
        v = sample_fn()
        lam = rng.uniform(-5, 5)

        # Condition 2: u + v should satisfy the same constraint as U
        # Condition 3: lam * u should satisfy the same constraint
        # We test these for two specific subspaces below.
        _ = lam * u + v # just compute; constraint checked per-example

    return failures == 0

# Subspace: U = {(x1, x2, x3) : x3 = 5*x1} (homogeneous -> subspace)
def sample_U():
    x1, x2 = rng.standard_normal(2)
    return np.array([x1, x2, 5 * x1])

def in_U(v, tol=1e-9):
    return abs(v[2] - 5 * v[0]) < tol

n_trials = 2000
u_fails = sum(
    not in_U(rng.uniform(-5,5) * sample_U() + sample_U())
    for _ in range(n_trials)
)
print(f"U (homogeneous): {u_fails}/{n_trials} failures -> subspace: {u_fails==0}")

# Non-subspace: W = {(x1, x2, x3) : x3 = 5*x1 + 1} (inhomogeneous)
def sample_W():
    x1, x2 = rng.standard_normal(2)
    return np.array([x1, x2, 5 * x1 + 1])

def in_W(v, tol=1e-9):
    return abs(v[2] - 5 * v[0] - 1) < tol

w_fails = sum(
    not in_W(rng.uniform(-5,5) * sample_W() + sample_W())
    for _ in range(n_trials)
)
```

```
print(f"W (inhomogeneous): {w_fails}/{n_trials} failures -> subspace: {w_fails==0}")
```

3 Span and Linear Independence

Remark 3.1 (Theory meets computation)**.** The abstract theory tells us *what* subspaces are. To actually *find* them — compute bases, check independence, determine whether a vector lies in a span — we need row reduction and matrix algorithms. From here onward, theory and computation go hand in hand.

§1.3 Linear combinations and span

Definition 3.1. A *linear combination* of vectors $v_1, \ldots, v_p \in V$ is any expression $\sum_{k=1}^{p} \lambda_k v_k$ with $\lambda_k \in \mathbb{F}$. It is *trivial* if all $\lambda_k = 0$.

The *span* of $\{v_1, \ldots, v_p\}$ is the set of all their linear combinations:

$$\operatorname{span}\{v_1, \ldots, v_p\} = \langle v_1, \ldots, v_p \rangle = \left\{ \sum_{k=1}^{p} \lambda_k v_k : \lambda_k \in \mathbb{F} \right\}.$$

A set $\{v_1, \ldots, v_p\}$ is a *generating* (or *spanning*) *system* for V if $\operatorname{span}\{v_1, \ldots, v_p\} = V$.

Proposition 3.1 (Span is the smallest containing subspace)**.** $\operatorname{span}\{v_1, \ldots, v_p\}$ *is a subspace of* V*, and it is the* smallest *subspace containing all of* $v_1, \ldots, v_p$*.*

Proof. It is a subspace. $0 = \sum_k 0 \cdot v_k \in \operatorname{span}$. If $u = \sum_k \lambda_k v_k$ and $w = \sum_k \mu_k v_k$, then $u + w = \sum_k (\lambda_k + \mu_k) v_k \in \operatorname{span}$ and $\alpha u = \sum_k (\alpha \lambda_k) v_k \in \operatorname{span}$.

It contains each v_i*.* Take $\lambda_i = 1$ and $\lambda_k = 0$ for $k \neq i$.

It is the smallest. Let $W \leq V$ contain each v_i. Any linear combination $\sum_k \lambda_k v_k$ belongs to W by closure, so $\operatorname{span}\{v_1, \ldots, v_p\} \subseteq W$. □

Example 3.1 (Span in $\mathbb{R}^2$)**.**

- $\operatorname{span}\{(1,0)^T, (0,1)^T\} = \mathbb{R}^2$: any $(a,b)^T = a(1,0)^T + b(0,1)^T$. These two vectors are linearly independent.
- $\operatorname{span}\{(1,1)^T, (2,2)^T\} = \{(t,t)^T : t \in \mathbb{R}\} \subsetneq \mathbb{R}^2$: both vectors point in the same direction, so their span is a line. These two vectors are linearly *dependent*: $(2,2)^T = 2(1,1)^T$.

Lesson: whether a set spans depends on linear independence, not just on the number of vectors.

Python: checking span via rank

```
import numpy as np

# Do v1, v2, v3 span R^3?
V = np.array([[1, 0, 1],
              [0, 1, 1],
              [0, 0, 0]], float) # columns are the vectors
rank = np.linalg.matrix_rank(V)
n = V.shape[0]
print("rank =", rank, " spans R^3:", rank == n)
```

```
# rank=2, so they do NOT span R^3

# Does a target vector b lie in the span?
b = np.array([1, 1, 0], float)
# Augment and check if rank increases
aug = np.column_stack([V, b])
print("b in span:", np.linalg.matrix_rank(aug) == rank)
```

Definition 3.2. A set of vectors $\{v_1, \ldots, v_p\}$ is *linearly dependent* if there exist scalars $\lambda_1, \ldots, \lambda_p$, *not all zero*, such that $\sum_{k=1}^{p} \lambda_k v_k = 0$.

The set is *linearly independent* if the only solution to $\sum_{k=1}^{p} \lambda_k v_k = 0$ is the trivial one ($\lambda_k = 0$ for all k).

Proposition 3.2 (Linear dependence $\iff$ one vector is redundant)**.** *A set $\{v_1, \ldots, v_p\} \subset V$ is linearly dependent if and only if there exists an index $k \in \{1, \ldots, p\}$ such that*

$$v_k = \sum_{\substack{j=1 \\ j \neq k}}^{p} \beta_j \, v_j.$$

Proof. ($\Rightarrow$) Suppose $\sum_{i=1}^{p} \lambda_i v_i = 0$ with some $\lambda_k \neq 0$. Then

$$\lambda_k v_k = -\sum_{j \neq k} \lambda_j v_j \implies v_k = \sum_{j \neq k} \left(-\frac{\lambda_j}{\lambda_k} \right) v_j.$$

($\Leftarrow$) If $v_k = \sum_{j \neq k} \beta_j v_j$, then

$$\sum_{j \neq k} \beta_j v_j + (-1) \cdot v_k = 0,$$

which is a nontrivial relation (the coefficient of v_k is $-1 \neq 0$). □

Example 3.2.

- In $\mathcal{P}_n$: the monomials $\{1, x, x^2, \ldots, x^n\}$ are linearly independent, since a polynomial identically equal to zero has all coefficients zero.
- In $\mathbb{R}^3$: the standard basis vectors $\{e_1, e_2, e_3\}$ are independent; $\{(1,0,0)^T, (2,0,0)^T\}$ are dependent since $(2,0,0)^T = 2(1,0,0)^T$.

Example 3.3 (Uniqueness of representation)**.** If $V = \langle v_1, \ldots, v_p \rangle$ and $\{v_1, \ldots, v_p\}$ is linearly independent, then every $x \in V$ has a *unique* representation $x = \sum_k \lambda_k v_k$.

Proof. If $\sum_k \lambda_k v_k = x = \sum_k \mu_k v_k$, then $\sum_k (\lambda_k - \mu_k) v_k = 0$. Linear independence forces $\lambda_k = \mu_k$ for all k. □

This uniqueness is exactly why we care about independent spanning sets (bases): they give a *coordinate system*.

4 Bases and Dimension

Definition 4.1 (Basis via direct sum)**.** A set $\{v_1, \ldots, v_n\} \subset V$ is a *basis* of V if

$$V = \langle v_1 \rangle \oplus \langle v_2 \rangle \oplus \cdots \oplus \langle v_n \rangle,$$

where $\langle v_i \rangle = \{\lambda v_i : \lambda \in \mathbb{F}\}$ is the one-dimensional subspace spanned by v_i.

Remark 4.1 (Coordinates)**.** The direct sum condition says exactly that every $v \in V$ has a *unique* representation

$$v = \lambda_1 v_1 + \lambda_2 v_2 + \cdots + \lambda_n v_n, \qquad \lambda_i \in \mathbb{F}.$$

The map

$$\varphi_B : V \longrightarrow \mathbb{F}^n, \qquad v \longmapsto (\lambda_1, \ldots, \lambda_n)^T$$

is an isomorphism — it identifies V with $\mathbb{F}^n$. Different bases give different isomorphisms, i.e. different coordinate systems. This is why all n-dimensional vector spaces over $\mathbb{F}$ are "the same": they are all isomorphic to $\mathbb{F}^n$.

Proposition 4.1 (Basis characterization — two definitions agree)**.** *The following are equivalent for a set $\{v_1, \ldots, v_n\} \subset V$:*

(i) $V = \langle v_1 \rangle \oplus \cdots \oplus \langle v_n \rangle$ *(basis via direct sum).*

(ii) $\{v_1, \ldots, v_n\}$ *is a linearly independent spanning set.*

(iii) $\{v_1, \ldots, v_n\}$ *is a* minimal *spanning set (removing any one vector destroys span).*

(iv) $\{v_1, \ldots, v_n\}$ *is a* maximal *linearly independent set (adding any vector creates dependence).*

Proof. (i) $\Longleftrightarrow$ (ii): The direct sum $V = \bigoplus_i \langle v_i \rangle$ means every $v \in V$ is uniquely $\sum_i \lambda_i v_i$. This is spanning. Uniqueness with $v = 0$ gives $\sum_i \lambda_i v_i = 0 \Rightarrow \lambda_i = 0$ for all i, which is linear independence. Conversely, independence + spanning gives uniqueness of representation, which is the direct sum condition.

(ii)$\Rightarrow$(iii): If some v_k can be removed and the rest still span, then $v_k \in \operatorname{span}\{v_j : j \neq k\}$, contradicting independence.

(iii)$\Rightarrow$(ii): A minimal spanning set must be independent (otherwise remove a redundant vector by the previous proposition and still span).

(ii)$\Rightarrow$(iv): Adding any $w \in V$ to the set: write $w = \sum \lambda_i v_i$ (by spanning); then $w - \sum \lambda_i v_i = 0$ is a nontrivial relation.

(iv)$\Rightarrow$(ii): Maximality forces every $w \in V$ to be in the span (otherwise $\{v_1, \ldots, v_n, w\}$ would be independent, contradicting maximality). $\square$

Python: coordinates in a basis

```
import numpy as np

# Basis vectors as columns of B
B = np.array([[1, 1, 0],
              [0, 1, 1],
              [1, 0, 1]], float)

v = np.array([3, 2, 1], float)
```

```python
# Coordinates of v in basis B: solve B @ coords = v
coords = np.linalg.solve(B, v)
print("Coordinates:", coords)

# Verify: reconstruct v from coordinates
print("Reconstructed:", B @ coords) # should equal v

# Different basis => different coordinates for the same vector
B2 = np.eye(3) # standard basis
coords2 = np.linalg.solve(B2, v)
print("Standard coords:", coords2) # just v itself
```

Python: differentiation as a matrix — and nilpotency

```python
import numpy as np

# Build the differentiation matrix D for P_n in the monomial basis
# {1, x, x^2, ..., x^n} (represented as coefficient vectors in R^{n+1})
# D(x^k) = k * x^{k-1}, D(1) = 0

def diff_matrix(n):
    """(n+1) x (n+1) matrix of D: P_n -> P_n in monomial basis."""
    D = np.zeros((n + 1, n + 1))
    for k in range(1, n + 1):
        D[k - 1, k] = k # coefficient of x^{k-1} in D(x^k) is k
    return D

D = diff_matrix(5)
print("D =\n", D.astype(int))

# Verify: D * [0,0,1,0,0,0]^T = coeff vector of D(x^2) = 2x = [0,2,0,0,0,0]
x2_coeffs = np.array([0, 0, 1, 0, 0, 0], float)
print("D(x^2) coeffs:", D @ x2_coeffs) # should be [0, 2, 0, 0, 0, 0]

# D is NILPOTENT: D^{n+1} = 0
print("D^6 = 0:", np.allclose(np.linalg.matrix_power(D, 6), 0))

# Show the staircase of powers
for k in range(1, 7):
    Dk = np.linalg.matrix_power(D, k)
    rank = np.linalg.matrix_rank(Dk)
    print(f" rank(D^{k}) = {rank}") # 5, 4, 3, 2, 1, 0

# Integrate back: J = right inverse of D. J(x^k) = x^{k+1}/(k+1)
def antideriv_matrix(n):
    """(n+2) x (n+1) matrix of J: P_n -> P_{n+1}."""
    J = np.zeros((n + 2, n + 1))
    for k in range(n + 1):
        J[k + 1, k] = 1.0 / (k + 1)
    return J

J = antideriv_matrix(5)
```

```
D6 = diff_matrix(6) # D acting on P_6 (to compose with J: P_5->P_6)
print("D o J = I:", np.allclose(D6 @ J, np.eye(6))) # True: FTC
```

Remark 4.2 (A basis makes everything computable)**.** Once a basis $\{v_1, \ldots, v_n\}$ of V is fixed, the coordinate isomorphism $\varphi_B : V \to \mathbb{F}^n$ turns every abstract vector into a concrete column of numbers, and every linear map into a matrix. The choice of basis is the choice of coordinate system; the mathematics is independent of that choice, but the computations depend on it. A good basis (e.g. an orthonormal one; see §3) makes computations fast and numerically stable.

§1.9 Matrices — a practical bridge

Example 4.1 (Digital images as matrices)**.** A **color image** $N = (R, G, B)$ is stored as three separate $m \times n$ arrays of pixel intensities, one per channel. Each entry is an integer in $\{0, \ldots, 255\}$.

A **grayscale image** collapses to a single $m \times n$ array:

$$A \in \{0, 1, \ldots, 255\}^{m \times n} \subset \mathbb{R}^{m \times n}.$$

A typical photograph might have $m = 3024$, $n = 4032$, giving over 12 million entries per channel — expensive to store or transmit raw.

Key observation. Natural images are not random matrices: rows and columns are highly correlated (nearby pixels look similar). This means the matrix A is *approximately low-rank*: most of its information lives in a small number of directions.

Forward pointer. The Singular Value Decomposition (SVD) gives the *best* low-rank approximation to any matrix:

$$A \approx A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T, \qquad k \ll \min(m, n).$$

JPEG compression, Netflix recommendations, and PCA in genomics all exploit this idea. We will prove the optimality of this approximation (Eckart–Young theorem) later in the course.

Python: images as matrices

```
import numpy as np
import matplotlib.pyplot as plt

# Create a synthetic grayscale "image" (or load a real one)
rng = np.random.default_rng(0)
m, n = 64, 64
# Low-rank signal + noise: true rank is 3
U = rng.standard_normal((m, 3))
V = rng.standard_normal((n, 3))
A = U @ V.T + 0.5 * rng.standard_normal((m, n))

# SVD
u, s, vt = np.linalg.svd(A, full_matrices=False)
print("Singular values (first 6):", np.round(s[:6], 2))

# Rank-k approximations and their errors
for k in [1, 3, 10]:
    Ak = u[:, :k] @ np.diag(s[:k]) @ vt[:k, :]
    err = np.linalg.norm(A - Ak, 'fro') / np.linalg.norm(A, 'fro')
    print(f" k={k:2d} relative Frobenius error: {err:.3f}")
```

Definition 4.2. A *matrix* over $\mathbb{F}$ is a rectangular array with m rows and n columns, with entries in $\mathbb{F}$:

$$A = (a_{ij})_{\substack{i=1,\dots,m \\ j=1,\dots,n}} \in \mathbb{F}^{m\times n}.$$

The (i,j)-entry of A is $(A)_{ij} = a_{ij}$. The j-th column of A is the vector $a_j = (a_{1j}, \dots, a_{mj})^T \in \mathbb{F}^m$.

Operations (defined entry-wise):

- Addition: $(A+B)_{ij} = a_{ij} + b_{ij}$.
- Scalar multiplication: $(\lambda A)_{ij} = \lambda\, a_{ij}$.

Proposition 4.2. *$\mathbb{F}^{m\times n}$ is a vector space over $\mathbb{F}$ of dimension mn.*

Proof. Vector space. Define the *vec* operation: stack the columns of A into a single vector,

$$\mathrm{vec}(A) = (a_{11}, \dots, a_{m1},\, a_{12}, \dots, a_{m2},\, \dots,\, a_{1n}, \dots, a_{mn})^T \in \mathbb{F}^{mn}.$$

Then vec is a bijection that respects addition and scalar multiplication, so $\mathbb{F}^{m\times n} \cong \mathbb{F}^{mn}$ as vector spaces. All 8 axioms are inherited entry-wise from $\mathbb{F}$.

Dimension and basis. Define the *standard matrix units* $E_{ij} \in \mathbb{F}^{m\times n}$ as the matrix with 1 in position (i,j) and 0 elsewhere. Every matrix decomposes uniquely as

$$A = \sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}\, E_{ij},$$

so $\{E_{ij}\}$ is a basis of $\mathbb{F}^{m\times n}$ with mn elements. Hence $\dim \mathbb{F}^{m\times n} = mn$. □

Definition 4.3 (Transpose)**.** The *transpose* of $A \in \mathbb{F}^{m\times n}$ is $A^T \in \mathbb{F}^{n\times m}$ defined by $(A^T)_{ij} = a_{ji}$. Equivalently, the columns of A^T are the rows of A, and vice versa.

Proposition 4.3 (Transpose properties)**.**

(a) $(A^T)^T = A$.

(b) $(\lambda A + \mu B)^T = \lambda A^T + \mu B^T$ *(linearity).*

(c) $(AB)^T = B^T A^T$ *(order reverses).*

Proof. (a) $((A^T)^T)_{ij} = (A^T)_{ji} = a_{ij}$.

(b) Entry-wise: $(\lambda A + \mu B)^T_{ij} = \lambda a_{ji} + \mu b_{ji} = (\lambda A^T + \mu B^T)_{ij}$.

(c) $((AB)^T)_{ij} = (AB)_{ji} = \sum_k a_{jk} b_{ki} = \sum_k (B^T)_{ik}(A^T)_{kj} = (B^T A^T)_{ij}$. □

Remark 4.3. Since the column space of A^T is the row space of A, the four fundamental subspaces of A come in two transpose-related pairs: $\mathrm{col}(A) \perp \ker(A^T)$ and $\mathrm{row}(A) \perp \ker(A)$.

Definition 4.4 (Matrix-vector product)**.** For $A \in \mathbb{F}^{m\times n}$ and $x \in \mathbb{F}^n$, the *matrix-vector product* $Ax \in \mathbb{F}^m$ is

$$Ax = \sum_{j=1}^{n} x_j\, a_j,$$

where a_j is the j-th column of A. That is, Ax is a *linear combination of the columns of A* with coefficients given by x.

Remark 4.4 (Column space interpretation)**.** The set $\{Ax : x \in \mathbb{F}^n\}$ is exactly $\mathrm{col}(A)$, the span of the columns of A. So $Ax = b$ has a solution iff $b \in \mathrm{col}(A)$.

Python: standard matrix units and vec

```python
import numpy as np

m, n = 3, 4

# Standard matrix unit E_{ij}
def E(i, j, m, n):
    M = np.zeros((m, n))
    M[i, j] = 1.0
    return M

# Decompose a matrix in the E_{ij} basis
A = np.random.randint(0, 9, (m, n)).astype(float)
A_reconstructed = sum(A[i, j] * E(i, j, m, n)
                      for i in range(m) for j in range(n))
print("Reconstruction error:", np.linalg.norm(A - A_reconstructed))

# vec operation: stack columns
vec_A = A.flatten(order='F') # column-major (Fortran order)
print("vec(A) shape:", vec_A.shape) # should be (m*n,)

# Matrix-vector product as linear combination of columns
x = np.array([1, 2, 0, -1], float)
Ax_direct = A @ x
Ax_lincombo = sum(x[j] * A[:, j] for j in range(n))
print("Ax via @: ", Ax_direct)
print("Ax via col combo: ", Ax_lincombo)
```

Matrix-matrix multiplication

Definition 4.5. For $A \in \mathbb{F}^{m\times n}$ and $B \in \mathbb{F}^{n\times k}$, the *matrix product* $AB \in \mathbb{F}^{m\times k}$ is defined by

$$(AB)_{ij} = \sum_{q=1}^{n} A_{iq}\, B_{qj}.$$

Note the dimensions must be compatible: the number of columns of A must equal the number of rows of B.

Proposition 4.4 (Three equivalent views of matrix multiplication)**.** *Let $b_1, \dots, b_k$ be the columns of B and $a_1^T, \dots, a_m^T$ the rows of A. Then:*

(i) ***Row-dot-product view:*** *$(AB)_{ij} = a_i^T b_j$ (row i of A dotted with column j of B).*

(ii) ***Column view:*** *the j-th column of AB is Ab_j (multiply A by each column of B separately):*

$$AB = \big[Ab_1 \mid Ab_2 \mid \cdots \mid Ab_k\big].$$

(iii) ***Outer-product (rank-1) view:***

$$AB = \sum_{q=1}^{n} a_q\, b_q^T,$$

where a_q is the q-th column of A and b_q^T is the q-th row of B. Each term $a_q b_q^T$ is a rank-1 matrix; AB is their sum.

Proof. All three follow directly from the definition.

(i) $(AB)_{ij} = \sum_q A_{iq}B_{qj} = a_i^T b_j$.

(ii) The j-th column of AB is the vector with i-th entry $(AB)_{ij} = \sum_q A_{iq}B_{qj} = (Ab_j)_i$.

(iii) $\sum_q (a_q b_q^T)_{ij} = \sum_q A_{iq}B_{qj} = (AB)_{ij}$. □

Remark 4.5 (Why is multiplication defined this way?)**.** Matrix multiplication is not an arbitrary formula — it is exactly *composition of linear maps*. If A represents a map $T_A : \mathbb{F}^n \to \mathbb{F}^m$ and B represents $T_B : \mathbb{F}^k \to \mathbb{F}^n$, then AB represents $T_A \circ T_B : \mathbb{F}^k \to \mathbb{F}^m$. The formula $(AB)_{ij} = \sum_q A_{iq}B_{qj}$ is forced by requiring $(T_A \circ T_B)(x) = T_A(T_B(x))$ for all x. This is why

- matrix multiplication is associative: $(AB)C = A(BC)$ (both equal $T_A \circ T_B \circ T_C$),
- but generally *not* commutative: $AB \neq BA$.

Python: three views of matrix multiplication

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]], float) # 2x3
B = np.array([[1, 0],
              [0, 1],
              [1, 1]], float) # 3x2

# Ground truth
AB = A @ B
print("AB =\n", AB)

# (i) Row-dot-product view
AB_row = np.array([[A[i] @ B[:, j] for j in range(B.shape[1])]
                   for i in range(A.shape[0])])
print("Row view matches:", np.allclose(AB, AB_row))

# (ii) Column view: AB[:,j] = A @ B[:,j]
AB_col = np.column_stack([A @ B[:, j] for j in range(B.shape[1])])
print("Column view matches:", np.allclose(AB, AB_col))

# (iii) Outer-product (rank-1) view
AB_outer = sum(np.outer(A[:, q], B[q, :]) for q in range(A.shape[1]))
print("Outer-product view matches:", np.allclose(AB, AB_outer))
```

5 Linear Maps

§1.5 Linear transformations

Remark 5.1 (A more natural view than matrices)**.** Matrices are powerful computational tools, but they depend on a choice of basis. Linear transformations are the *basis-free* concept that matrices represent. The hierarchy is:

$$\text{set map} \subset \text{linear map (between vector spaces)} \subset \text{matrix (linear map in chosen bases)}.$$

Understanding the linear map first, and the matrix second, is the more natural order.

Definition 5.1 (Transformation)**.** A *transformation* (or *map*, or *function*) from a set X to a set Y is a rule T that assigns to each $x \in X$ exactly one element $T(x) \in Y$. We write $T : X \to Y$.

No linearity is assumed yet — X and Y are just sets.

Definition 5.2 (Linear map)**.** Let V, W be vector spaces over $\mathbb{F}$. The set V is called the *domain* of T and W the *target space* (or *codomain*). We write $T : V \to W$.

T is *linear* if for all $u, v \in V$ and $\lambda, \mu \in \mathbb{F}$:

1. $T(u + v) = T(u) + T(v)$ *(additivity)*,
2. $T(\lambda v) = \lambda T(v)$ *(homogeneity)*,

or equivalently: $T(\lambda u + \mu v) = \lambda T(u) + \mu T(v)$.

Example 5.1 (Linear maps in the wild)**.**

1. **Matrix multiplication.** For $A \in \mathbb{F}^{m\times n}$, the map $T : \mathbb{F}^n \to \mathbb{F}^m$, $T(x) = Ax$, is linear.

   Proof. $T(\lambda u + \mu v) = A(\lambda u + \mu v) = \lambda Au + \mu Av = \lambda T(u) + \mu T(v)$. □

   Every linear map between finite-dimensional spaces is of this form (once bases are chosen).

2. **Differentiation.** Define $D : \mathcal{P}_n \to \mathcal{P}_n$ by $D(p) = p'$. Linearity follows from the linearity of the derivative: $D(\lambda p + \mu q) = (\lambda p + \mu q)' = \lambda p' + \mu q' = \lambda D(p) + \mu D(q)$.

   On the monomial basis $\{1, x, x^2, \dots, x^n\}$:

   $$D(1) = 0, \quad D(x) = 1, \quad D(x^2) = 2x, \quad \dots, \quad D(x^k) = kx^{k-1}.$$

   Writing coordinates as column vectors $(a_0, a_1, \dots, a_n)^T$, the matrix of D in the monomial basis is the $(n + 1) \times (n + 1)$ upper-shift matrix:

   $$[D] = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 2 & \cdots & 0 \\ \vdots & & & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & n \\ 0 & 0 & 0 & \cdots & 0 \end{pmatrix}.$$

3. **Integration (exercise).** Define $J : \mathcal{P}_n \to \mathcal{P}_{n+1}$ by

   $$J(p)(x) = \int_0^1 x\, p(xt)\, dt.$$

   Note: by the substitution $u = xt$, this equals $\int_0^x p(t)\, dt$ (the antiderivative vanishing at 0).

   Exercise 5.1.

   (a) Show that J is linear.

   (b) Compute $J(x^k)$ for $k = 0, 1, \dots, n$ and write the matrix of J in the monomial bases of $\mathcal{P}_n$ and $\mathcal{P}_{n+1}$.

   (c) Show that $D \circ J = \mathrm{id}_{\mathcal{P}_n}$ (J is a right inverse of D).

   (d) Is $J \circ D = \mathrm{id}_{\mathcal{P}_{n+1}}$? If not, what goes wrong, and what is $J \circ D$ instead?

(e) Interpret parts (c) and (d) as the Fundamental Theorem of Calculus.

Proposition 5.1 (A linear map is determined by its values on a basis)**.** *Let $\{v_1, \dots, v_n\}$ be a basis of V and let $w_1, \dots, w_n \in W$ be arbitrary. Then there exists a* unique *linear map $T : V \to W$ such that $T(v_i) = w_i$ for $i = 1, \dots, n$.*

Proof. Existence. Every $v \in V$ has a unique representation $v = \sum_i \lambda_i v_i$. Define $T(v) = \sum_i \lambda_i w_i$. This is well-defined (uniqueness of coordinates) and linear:

$$T(\lambda u + \mu v) = T\Big(\sum_i (\lambda\alpha_i + \mu\beta_i)v_i\Big) = \sum_i (\lambda\alpha_i + \mu\beta_i)w_i = \lambda T(u) + \mu T(v).$$

Uniqueness. If $S : V \to W$ is also linear with $S(v_i) = w_i$, then for any $v = \sum_i \lambda_i v_i$:

$$S(v) = \sum_i \lambda_i S(v_i) = \sum_i \lambda_i w_i = T(v). \qquad \square$$

Remark 5.2. This proposition is the engine behind matrix representations: to specify a linear map $T : \mathbb{F}^n \to \mathbb{F}^m$, it suffices to say where the n basis vectors go. The resulting matrix is $[T] = [T(e_1) \mid T(e_2) \mid \cdots \mid T(e_n)]$.

Python: constructing a linear map from its action on a basis

```
import numpy as np

# Define T: R^3 -> R^2 purely by saying where the basis vectors go.
# T(e1) = (1, 2), T(e2) = (-1, 0), T(e3) = (3, -1)
T_e1 = np.array([1., 2.])
T_e2 = np.array([-1., 0.])
T_e3 = np.array([3., -1.])

# The matrix of T is [T(e1) | T(e2) | T(e3)]
M_T = np.column_stack([T_e1, T_e2, T_e3])
print("Matrix of T:\n", M_T)

# Apply T to an arbitrary vector by decomposing in the standard basis
v = np.array([2., -3., 1.])
# T(v) = 2*T(e1) + (-3)*T(e2) + 1*T(e3) (by linearity)
T_v_direct = v[0]*T_e1 + v[1]*T_e2 + v[2]*T_e3
T_v_matrix = M_T @ v
print("T(v) via linearity:", T_v_direct)
print("T(v) via M_T @ v: ", T_v_matrix)
print("Same?", np.allclose(T_v_direct, T_v_matrix))

# Now define T on a NON-standard basis B = {b1, b2, b3} of R^3
B = np.array([[1, 1, 0],
             [0, 1, 1],
             [1, 0, 1]], float) # columns are b1, b2, b3
# Specify T(b1), T(b2), T(b3)
T_B = np.array([[2., 0., -1.],
               [1., 3., 0.]]) # columns are T(b_i)

# Matrix of T in standard basis: M_T = T_B @ B^{-1}
```

```
M_T_standard = T_B @ np.linalg.inv(B)
print("\nMatrix of T in standard basis (from non-standard spec):\n",
     np.round(M_T_standard, 4))
```

Matrix representation of a linear map

Proposition 5.2. *Let $T : \mathbb{F}^n \to \mathbb{F}^m$ be linear and $\{e_1, \dots, e_n\}$ the standard basis of $\mathbb{F}^n$. Define*

$$M_T = \big[T(e_1) \mid T(e_2) \mid \cdots \mid T(e_n)\big] \in \mathbb{F}^{m\times n}.$$

Then $T(v) = M_T v$ for all $v \in \mathbb{F}^n$.

Proof. Write $v = \sum_{i=1}^n v_i e_i$. By linearity and the column view of matrix-vector multiplication:

$$T(v) = \sum_{i=1}^n v_i\, T(e_i) = M_T v. \qquad \square$$

Remark 5.3. The i-th column of M_T is $T(e_i)$: apply T to each standard basis vector, stack the results as columns.

$\mathcal{L}(V, W)$ is a vector space

Proposition 5.3. *The set $\mathcal{L}(V, W)$ of all linear maps $T : V \to W$ is a vector space under pointwise operations: $(S + T)(v) = S(v) + T(v)$ and $(\lambda T)(v) = \lambda T(v)$.*

Proof. Closure under addition: $(S + T)(\lambda u + \mu v) = S(\lambda u + \mu v) + T(\lambda u + \mu v) = \lambda(S + T)(u) + \mu(S + T)(v)$, so $S + T$ is linear. Similarly λT is linear. The zero map $0 : v \mapsto 0_W$ is the additive identity; all eight axioms are inherited pointwise from W. $\square$

Remark 5.4. Under $T \leftrightarrow M_T$, we get $\mathcal{L}(\mathbb{F}^n, \mathbb{F}^m) \cong \mathbb{F}^{m\times n}$ as vector spaces, which is why matrix operations are defined entry-wise.

Composition = matrix multiplication

Theorem 5.1. *Let $T : \mathbb{F}^n \to \mathbb{F}^m$ with matrix $A = M_T$ and $S : \mathbb{F}^m \to \mathbb{F}^k$ with matrix $B = M_S$. Then $S \circ T : \mathbb{F}^n \to \mathbb{F}^k$ is linear with*

$$M_{S\circ T} = BA.$$

Proof. Let $a_i = T(e_i)$ be the i-th column of A. For any $u = \sum_i u_i e_i \in \mathbb{F}^n$:

$$(S \circ T)(u) = S\Big(\sum_i u_i a_i\Big) = \sum_i u_i S(a_i) = \sum_i u_i (Ba_i) = B\Big(\sum_i u_i a_i\Big) = B(Au) = (BA)u. \qquad \square$$

Remark 5.5. Associativity of matrix multiplication ($A(BC) = (AB)C$) is now immediate: it is just associativity of composition of functions.

Python: composition and matrix multiplication

```
import numpy as np

A = np.array([[2, 1],
              [0, 3]], float) # matrix of T
B = np.array([[1, -1],
              [2, 0]], float) # matrix of S

x = np.array([1, 2], float)

# S(T(x)) = (BA)x
print("S(T(x)) =", B @ (A @ x))
print("(BA)x =", (B @ A) @ x) # same

# Build M_T by applying T to each standard basis vector
n = 2
M_T = np.column_stack([A @ np.eye(n)[:, i] for i in range(n)])
print("M_T == A:", np.allclose(M_T, A))
```

Remark 5.6 (Summary: what matrix operations mean)**.**

- **Matrix-vector multiplication** = applying a linear transformation: $Av = T_A(v)$.
- **Matrix-matrix multiplication** = composing linear transformations: $BA = M_{S \circ T}$.

Every calculation with matrices is secretly a calculation with linear maps. The matrix is just the coordinate representation; the map is the intrinsic object.

Many questions — the roadmap ahead

The theory so far raises three natural questions that will drive the rest of the course.

1. **Any basis, not just $e_1, \ldots, e_n$.** We built M_T using the standard basis. Given a different basis $\{b_1, \ldots, b_n\}$, can we write a matrix for T? How do the two matrices relate? *This leads to: change of basis and similarity ($A_B = P^{-1}AP$).*
2. **Invertibility.** A general function $f : X \to Y$ is invertible iff it is a bijection. When is a linear map invertible? When is it *not*, and what structure does the failure have? *This leads to: kernel, image, rank–nullity theorem, and the four fundamental subspaces.*
3. **Good examples of linear transformations.** The identity and zero map are trivial. What are the interesting, structurally rich examples? Can we always find a basis in which a linear map looks simple? *This leads to: eigenvalues, diagonalization, the spectral theorem, and the SVD.*

6 Column Space, Null Space, and Rank–Nullity

§1.6 Matrix algorithms

Let $A \in \mathbb{R}^{n \times m}$.

The columns of A span a subspace of $\mathbb{R}^n$ called the *image* of A.

$$L_A(x) = Ax : \mathbb{R}^m \longrightarrow \mathbb{R}^n \quad \text{is a linear transform.}$$

Definition 6.1.

$$\mathrm{im}_A = \mathrm{im}\, A = \{Ax : x \in \mathbb{R}^m\} = \big\langle (a_{1i},\, a_{2i},\, \ldots,\, a_{ni})^T,\ i = 1, \ldots, m \big\rangle.$$

The columns of A may form a basis, or have non-trivial linear relations. In the latter case,

$$\lambda_1 \begin{pmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{n1} \end{pmatrix} + \cdots = \sum_{i=1}^{m} \lambda_i \begin{pmatrix} a_{1i} \\ a_{2i} \\ \vdots \\ a_{ni} \end{pmatrix} = 0 \tag{1}$$

for some $(\lambda_1, \ldots, \lambda_m)^T \neq 0$. Equation (1) can be written as

$$Av = 0, \qquad v = (\lambda_1, \lambda_2, \ldots, \lambda_m)^T.$$

Definition 6.2.

$$\ker_{L_A} = \ker A = \{x \in \mathbb{R}^m : Ax = 0\}.$$

Proposition 6.1 (1.6.1)**.** $\ker A \leq \mathbb{R}^m$ *and* $\mathrm{im}\, A \leq \mathbb{R}^n$.

Proof. Kernel. $A \cdot 0 = 0$, so $0 \in \ker A$. If $Ax = 0$ and $Ay = 0$ then $A(x + y) = Ax + Ay = 0$ and $A(\lambda x) = \lambda Ax = 0$, so $\ker A$ is closed under addition and scalar multiplication. By the one-condition subspace test, $\ker A \leq \mathbb{R}^m$.

Image. $A \cdot 0 = 0 \in \mathrm{im}\, A$. If $u = Ax$ and $w = Ay$ then $u + w = A(x + y) \in \mathrm{im}\, A$ and $\lambda u = A(\lambda x) \in \mathrm{im}\, A$. Hence $\mathrm{im}\, A \leq \mathbb{R}^n$. □

Python: kernel as the space of linear relations

```
import numpy as np

# Columns 1, 2, 3 satisfy: -col1 - col2 + col3 = 0
A = np.array([[1, 2, 3],
              [2, 4, 6]], float) # rank 1: col2 = 2*col1, col3 = 3*col1

v = np.array([-1, -1, 1]) # coefficients of the relation
print("Av =", A @ v) # -> [0. 0.]

# Verify via null space (scipy)
from scipy.linalg import null_space
K = null_space(A)
print("Basis for ker A:\n", K) # 2-dimensional: two free variables
```

Column operations and echelon form

Since $\mathrm{im}\, A$ is spanned by the columns of A and is closed under linear combinations, *column operations that replace a column by a linear combination of columns do not change* $\mathrm{im}\, A$. Concretely, two operation types preserve the image:

1. $\bar{a}_j \leftarrow \bar{a}_j + \lambda \bar{a}_i$, or $\bar{a}_i \leftarrow \lambda \bar{a}_i$ $(\lambda \neq 0)$.
2. Column swap: $i \leftrightarrow j$.

Why $\operatorname{im} A' = \operatorname{im} A$. Every column of A' is a linear combination of columns of A, so $\operatorname{im} A' \subseteq \operatorname{im} A$. Since each operation is invertible (the inverse is the same type), the reverse inclusion holds too. $\square$

Applying these operations repeatedly brings A to *echelon reduced form* A', with $\operatorname{im} A = \operatorname{im} A'$.

Algorithm (step 1).

1. Find the first column with a non-zero entry; take the bottom non-zero entry as the pivot.
2. Swap it into the leading position; scale to make it 1.
3. Use operation (1) to zero out all other entries in that row.
4. Repeat on the remaining submatrix (next column, next row).

Example 6.1.

$$A = \begin{pmatrix} 4 & 1 & 6 & 4 \\ 2 & 0 & 1 & 7 \\ 2 & 1 & 2 & 0 \end{pmatrix} \longrightarrow \cdots \longrightarrow \begin{pmatrix} -6 & 1 & 4 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

The pivot columns of the reduced form give a **basis for** $\operatorname{im} A$; the remaining (zero) columns point toward $\ker A$ (see page 17).

Remark 6.1 (Row vs. column operations)**.** These notes apply column operations directly to A (natural, since $\operatorname{im} A$ is about columns). Most textbooks instead apply *row* operations to A^T, which is equivalent. The resulting pivot structure is the same; only the notation differs.

Python: column reduction and basis for the image

```
import numpy as np

A = np.array([[4, 1, 6, 4],
              [2, 0, 1, 7],
              [2, 1, 2, 0]], float)

# Rank and pivot columns via QR with column pivoting
Q, R, P = np.linalg.qr(A.T, mode='complete') # QR on A^T
rank = np.linalg.matrix_rank(A)
print("rank =", rank) # 3

# The first `rank` columns of A (reordered by pivoting) span im A
basis = A[:, P[:rank]]
print("basis for im A:\n", basis)

# Quick check: the original columns live in the span
coeffs, res, _, _ = np.linalg.lstsq(basis, A, rcond=None)
print("residual (should be ~0):", res)
```

Combined algorithm for $\operatorname{im} A$ and $\ker A$

Definition 6.3. $I_{n\times n} = I$ is the $n \times n$ *identity matrix*: 1s on the diagonal, 0s elsewhere.

Augment $A \in \mathbb{R}^{n\times m}$ with I_m below it and apply column operations to the *whole* matrix simultaneously:

$$\left(\begin{array}{c} A \\ \hline I_m \end{array}\right) \xrightarrow{\text{column-reduce top block}} \left(\begin{array}{cc} \text{pivots} & \mathbf{0} \\ \hline t & * \end{array}\right).$$

At the end:

- **Non-zero columns** of the top block $\longrightarrow$ basis for $\operatorname{im} A$.
- **Zero columns** of the top block $\longrightarrow$ corresponding columns of the bottom block form a basis for $\ker A$.

Why the bottom block gives $\ker A$*.* Each column operation on the full matrix accumulates in the bottom block. After all operations, the j-th bottom column is a vector $v_j \in \mathbb{R}^m$ such that Av_j equals the j-th top column. If the j-th top column is $\mathbf{0}$, then $Av_j = 0$, so $v_j \in \ker A$. These v_j are linearly independent (they are columns of an invertible accumulated transformation), so they form a basis for $\ker A$. $\square$

Example 6.2.

$$\left(\begin{array}{c} A \\ \hline I_4 \end{array}\right) = \left(\begin{array}{cccc} 4 & 1 & 6 & 4 \\ 2 & 0 & 1 & 7 \\ 2 & 1 & 2 & 0 \\ \hline 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array}\right) \longrightarrow \cdots \longrightarrow \left(\begin{array}{c|c} \operatorname{im} A & \mathbf{0} \\ \hline t & \ker A \end{array}\right).$$

The three non-zero top columns give a basis for $\operatorname{im} A$; the one zero top column gives a basis vector for $\ker A$.

Remark 6.2 (Basis non-uniqueness)**.** Different orderings of column operations yield different bases for $\ker A$ and $\operatorname{im} A$. The *subspaces* themselves are canonical; only the chosen bases vary.

Remark 6.3 (Special case: matrix inverse)**.** When A is square ($n = m$) and full rank, $\ker A = \{0\}$, and the algorithm reduces the top block to I. The accumulated bottom block is then exactly A^{-1} — this is *Gauss–Jordan elimination.* The combined im/ker algorithm is the general case; matrix inversion is the special case where the kernel is trivial.

Python: image and kernel via `scipy`

```
import numpy as np
from scipy.linalg import orth, null_space

A = np.array([[4, 1, 6, 4],
              [2, 0, 1, 7],
              [2, 1, 2, 0]], float)

im_basis = orth(A) # a basis for im A
ker_basis = null_space(A) # a basis for ker A

print("dim im A =", im_basis.shape[1]) # 3
```

```
print("dim ker A =", ker_basis.shape[1]) # 1
print("dims sum to m =",
      im_basis.shape[1] + ker_basis.shape[1]) # 4 = m (rank-nullity preview)

# Verify: ker vectors map to 0
print("A @ ker_basis =\n", A @ ker_basis) # ~0
```

Remark 6.4. `scipy.linalg.orth` and `null_space` return bases whose columns happen to be mutually perpendicular and unit-length (*orthonormal*). This is a computational convenience, not a requirement: any basis for $\operatorname{im} A$ or $\ker A$ would do. Orthonormality will be defined properly in §3.

The algorithm of pages 16–17 produces the block structure

basis of $\operatorname{im} A$	$\mathbf{0}$
t	basis of $\ker A$

If dim = #basis vectors, this directly reads off $\dim \operatorname{im} A$ and $\dim \ker A$.

Theorem 6.1 (1.6.1 — Rank–Nullity)**.** *For any $A \in \mathbb{R}^{m\times n}$:*

$$\dim \operatorname{im} A + \dim \ker A = n.$$

Remark 6.5 (Logical order)**.** The theorem is stated here, but its proof requires knowing that dim is *well-defined* — i.e. that every basis of a given subspace has the same number of vectors. This is not obvious; we establish it via a rank argument before giving the proof.

Rank and well-definedness of dimension

Let B_1 be a basis of V and B_2 a linearly independent set $B_2 \subseteq V$.

Definition 6.4 (Rank)**.** For $A = [a_1\ a_2\ \cdots\ a_n] \in \mathbb{R}^{m\times n}$,

$$\operatorname{rk} A = \#\text{ the largest set of linearly independent columns of } A.$$

Remark 6.6 (Two equivalent formulations of rank)**.** The definition above (largest linearly independent subset of columns) agrees with the algorithmic one: $\operatorname{rk} A$ equals the number of pivot columns produced by the column-reduction algorithm of page 16. Both descriptions define the same number; the algorithmic one is how you *compute* it, the set-theoretic one is what it *means*.

Key question. If B_1 has n_1 vectors and B_2 has n_2 vectors, what is $\operatorname{rk}[B_1 \mid B_2]$? *(Continued on page 19.)*

Python: rank is independent of the chosen basis

```
import numpy as np

# Two different bases for the same plane in R^3 (the xy-plane)
B1 = np.array([[1, 0],
               [0, 1],
               [0, 0]], float)
```

```
B2 = np.array([[1, 1],
               [1, 0],
               [0, 0]], float) # different basis, same subspace

print(np.linalg.matrix_rank(B1)) # 2
print(np.linalg.matrix_rank(B2)) # 2 -- same rank regardless of basis
```

Remark 6.7 (Steinitz exchange lemma)**.** The rank argument on pages 18–19 is essentially the *Steinitz exchange lemma*: any linearly independent set can be extended to a basis, and no linearly independent set can be larger than any spanning set. This is the standard route to proving that dimension is well-defined. See any reference on linear algebra for the full statement.

Rank sandwich and well-definedness of dimension

Continuing from page 18: let B_1 be a basis of V with n_1 vectors and B_2 a linearly independent set in V with n_2 vectors. Then:

$$\begin{aligned} \operatorname{rk}[B_1 \mid B_2] &\geq n_2 && (B_2 \text{ is l.i., so those } n_2 \text{ columns of } [B_1 \mid B_2] \text{ are independent}) \\ \operatorname{rk}[B_1 \mid B_2] &\leq n_1 && (B_1 \text{ spans } V\text{, so every column of } B_2 \text{ is a lin. comb. of } B_1) \end{aligned}$$

Hence

$$n_2 \;\leq\; \operatorname{rk}[B_1 \mid B_2] \;\leq\; n_1.$$

If B_1 and B_2 are *both* bases of V, the roles are interchangeable: $n_2 \leq n_1$ and $n_1 \leq n_2$, so $\boxed{n_1 = n_2}$. Applying this to $V = \operatorname{im} A$: if B_1 and B_2 are any two bases for $\operatorname{im} A$, the sandwich gives $n_1 = n_2$, so $\operatorname{rk} A = \dim \operatorname{im} A$ is independent of the chosen basis.

Theorem 6.2 (1.6.2 — Dimension is well-defined)**.** $\dim V = \#$*basis vectors of* V *is independent of the chosen basis.*

Corollary 6.1 (1.6.3)**.**

- ***Matrix form (Rank-Nullity).*** *For any* $A \in \mathbb{R}^{m \times n}$*:*

$$\operatorname{rk} A + \dim \ker A = n.$$

- ***Abstract form (Im-Ker Theorem).*** *For any linear map* $L : V \to W$*:*

$$\dim \operatorname{im} L + \dim \ker L = \dim V.$$

Rank-Nullity *is the standard name in undergraduate texts and applies to matrices, where rank and nullity are well-defined numbers.* Im-Ker Theorem *is the coordinate-free version: it lives at the level of linear maps between abstract vector spaces, where "rank" is not directly defined but* $\dim \operatorname{im} L$ *is.*

Remark 6.8 (Two routes to the same result)**.** The notes prove dimension is well-defined via a *rank sandwich* argument (comparing $\operatorname{rk}[B_1 \mid B_2]$ from two sides). Most textbooks take a different but equivalent route via the *Steinitz exchange lemma* directly. Both establish the same fact; the rank-sandwich approach is particularly natural here because rank has already been defined algorithmically.

Python: rank-nullity numerically

```
import numpy as np
rng = np.random.default_rng(42)

A = rng.standard_normal((5, 8)) # generic 5x8 matrix
r = np.linalg.matrix_rank(A)
null_dim = 8 - r # dim ker A = n - rank

print(f"rank = {r}, nullity = {null_dim}, sum = {r + null_dim}")
# rank = 5, nullity = 3, sum = 8 = n (always)

# Also works for rank-deficient matrices
B = np.zeros((5, 8))
B[:2, :2] = np.eye(2) # rank 2
r2 = np.linalg.matrix_rank(B)
print(f"rank = {r2}, nullity = {8 - r2}, sum = {r2 + (8 - r2)}")
# rank = 2, nullity = 6, sum = 8
```

Finding coordinates and solving $Ax = b$

Problem. Given a basis $B = \{b_1, \ldots, b_n\}$ of V and a vector $v \in V$, find the coordinates of v in B.

Algorithm. Apply column operations to $[B \mid v]$ to zero out v, tracing the same operations on $[I \mid 0]$:

$$\left(\begin{array}{c|c} B & v \\ \hline I & 0 \end{array}\right) \xrightarrow{\text{col. ops on } [B|v]\text{, traced on } [I|0]} \left(\begin{array}{c|c} B & 0 \\ \hline I & w \end{array}\right).$$

The operations express $v = -\sum_{i=1}^{n} u_i b_i$, accumulating the coefficient vector $u = (u_1, \ldots, u_n)^T$ in the bottom-right block.

Solution: $v = -\displaystyle\sum_{i=1}^{n} u_i\, b_i$, so u (up to sign) gives the coordinates of v in basis B.

Remark 6.9 (Same algorithm, three uses)**.** The coordinate-finding algorithm is the combined im/ker algorithm of page 17 with v playing the role of an extra column to be eliminated. One idea — column-reduce while tracing on an identity — does three jobs: finding a basis for $\operatorname{im} A$, finding a basis for $\ker A$, and finding coordinates in a basis.

More generally: solving $Ax = b$.

$$Ax = b \iff \left(\begin{array}{c|c} A & -b \\ \hline I & 0 \end{array}\right) \xrightarrow{\text{col. reduce}} \left(\begin{array}{c|c} A & 0 \\ \hline \cdot & x \end{array}\right) \quad \text{(Gaussian elimination).}$$

The last column of the bottom block gives x.

Remark 6.10 (Uniqueness of solution)**.** If $\ker A \neq \{0\}$, the algorithm finds *one* particular solution x_0; the complete solution set is $x_0 + \ker A$ (a coset, not a subspace). Different column-operation orderings may yield different particular solutions, all differing by a vector in $\ker A$.

Remark 6.11 (When $b \notin \operatorname{im} A$)**.** The algorithm fails: the last column cannot be zeroed out. This is exactly the situation in polynomial regression (§2): the vector of observed values y is generally not in the column space of the feature matrix X, so exact solution is impossible and we instead minimise $\|Ax - b\|$. The least-squares problem is the natural generalisation.

Python: solving $Ax = b$ via numpy

```
import numpy as np

A = np.array([[1, 2],
              [3, 4],
              [5, 6]], float)
b = np.array([1., 1., 1.])

# Exact solve when b in im A
b_exact = A @ np.array([1., -1.]) # construct b that IS in im A
x_exact, _, _, _ = np.linalg.lstsq(A, b_exact, rcond=None)
print("exact solution:", x_exact) # [1. -1.]
print("residual:", np.linalg.norm(A @ x_exact - b_exact)) # ~0

# When b is not in im A: lstsq gives least-squares solution
x_ls, _, _, _ = np.linalg.lstsq(A, b, rcond=None)
print("least-squares solution:", x_ls)
print("residual:", np.linalg.norm(A @ x_ls - b)) # > 0
```

The three outcomes of solving $Ax = b$

If $b \in \operatorname{im} A$, column-reducing $\left(\begin{smallmatrix} A & -b \\ I & 0 \end{smallmatrix}\right)$ yields:

$$\begin{pmatrix} A & -b \\ I & 0 \end{pmatrix} \longrightarrow \begin{pmatrix} I & 0 \\ A^{-1} & x \end{pmatrix} \qquad (\text{when } \ker A = \{0\}).$$

The complete picture is a 2×2 table:

	$b \in \operatorname{im} A$	$b \notin \operatorname{im} A$
$\ker A = \{0\}$	unique solution $x = A^{-1}b$	no solution (vacuous if A square invertible)
$\ker A$ non-trivial	infinitely many: $x_0 + \ker A$	no solution

Definition 6.5 (Matrix inverse)**.** If $\ker A = \{0\}$ and A is square ($n = m$), define $B = A^{-1}$ by

$$AB = BA = I.$$

Remark 6.12 (Invertibility condition)**.** A^{-1} exists if and only if A is square and $\ker A = \{0\}$. For a square A, the Im-Ker Theorem gives the equivalences:

$$\ker A = \{0\} \iff \operatorname{rk} A = n \iff \operatorname{im} A = \mathbb{R}^n.$$

Injectivity and surjectivity coincide for square matrices — a fact with no analogue for infinite-dimensional spaces.

Remark 6.13 (Connection to Gauss–Jordan)**.** The case $\ker A = \{0\}$ is exactly the page-17 combined algorithm applied to a square full-rank matrix: $[A \mid I] \to [I \mid A^{-1}]$. Computing A^{-1} is not a separate algorithm — it is the im/ker algorithm in the special case where the kernel is trivial.

Python: the three cases and matrix inversion

```python
import numpy as np
from scipy.linalg import null_space

# Case 1: ker A = {0}, b in im A -> unique solution
A = np.array([[1., 2.], [3., 4.]])
b = A @ np.array([1., -1.]) # b is in im A by construction
x = np.linalg.solve(A, b)
print("unique solution:", x) # [1. -1.]

# Verify AB = BA = I
Ai = np.linalg.inv(A)
print("AB = I:", np.allclose(A @ Ai, np.eye(2))) # True
print("BA = I:", np.allclose(Ai @ A, np.eye(2))) # True

# Case 2: ker A non-trivial, b in im A -> infinitely many solutions
S = np.array([[1., 2., 3.],
             [4., 5., 6.]], float) # rank 2, ker non-trivial
b2 = S @ np.array([1., 0., 0.]) # b2 in im S
x0, _, _, _ = np.linalg.lstsq(S, b2, rcond=None) # one particular solution
K = null_space(S) # basis for ker S
print("particular solution:", x0)
print("x0 + ker S also solves:", np.allclose(S @ (x0 + K[:, 0]), b2)) # True

# Case 3: b not in im A -> no solution (residual > 0)
b3 = np.array([1., 1.])
x3, _, _, _ = np.linalg.lstsq(S, b3, rcond=None)
print("residual:", np.linalg.norm(S @ x3 - b3)) # > 0
```

Orthogonal complement and the four fundamental subspaces

Going back to the image-kernel theorem:

$$\begin{aligned} \operatorname{im} A &= \operatorname{col}(A) = \text{column space of } A, \\ \operatorname{im}(A^T) &= \operatorname{row}(A) = \text{row space of } A, \\ \ker A &= \operatorname{null}(A) = \text{null space of } A, \\ \ker(A^T) &= \operatorname{null}(A^T) = \text{left null space of } A. \end{aligned}$$

For each subspace $U \leq V$ (with inner product $\langle \cdot, \cdot \rangle$), define the *orthogonal complement*

$$U^\perp = \{w \in V : \langle w, u \rangle = 0 \ \forall\, u \in U\}.$$

Proposition 6.2 (Homework)**.** $V = U \oplus U^\perp$.

Taking $U = \operatorname{row}(A) \leq \mathbb{R}^n$:

$$\mathbb{R}^n = \operatorname{row}(A) \oplus \ker(A), \qquad n = \dim \operatorname{row}(A) + \dim \ker(A).$$

This gives $\operatorname{rank}(A) = \dim \operatorname{row}(A)$.

Theorem 6.3. *Column rank of A = row rank of A = $\operatorname{rk}(A)$.*

Python example

```
import numpy as np

A = np.array([[1, 2, 3],
              [2, 4, 6],
              [1, 1, 1]], float)
U, s, Vt = np.linalg.svd(A)
rank = np.sum(s > 1e-10)
nullity = A.shape[1] - rank
print("rank =", rank, " nullity =", nullity,
      " sum =", rank + nullity) # should equal n=3

# Null space basis: right singular vectors for s~0
null_basis = Vt[rank:].T
print("Null space basis:\n", null_basis)
print("A @ null_basis:\n", A @ null_basis) # should be ~0
```

7 Application: Hamming(7,4) Error-Correcting Code

The Hamming(7,4) code is one of the oldest and most elegant applications of linear algebra over $\mathbb{Z}_2$. It encodes 4-bit messages into 7-bit codewords in a way that allows the receiver to detect *and correct* any single-bit transmission error. Everything — encoding, error detection, and correction — reduces to kernel and image.

Setup: the parity-check matrix

Work over the field $\mathbb{Z}_2 = \{0, 1\}$ with addition mod 2. Define the *parity-check matrix*

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \in \mathbb{Z}_2^{3\times 7}.$$

The j-th column of H is the 3-bit binary representation of j (for $j = 1, \ldots, 7$).

Definition 7.1 (Linear code)**.** The *Hamming(7,4) code* is the kernel of H over $\mathbb{Z}_2$:

$$\mathcal{C} = \ker(H) = \{c \in \mathbb{Z}_2^7 : Hc = 0\}.$$

Elements of $\mathcal{C}$ are called *codewords*.

By the rank-nullity theorem (over $\mathbb{Z}_2$):

$$\dim \ker(H) = 7 - \operatorname{rank}(H) = 7 - 3 = 4,$$

so $|\mathcal{C}| = 2^4 = 16$: exactly 16 codewords, one for each 4-bit message.

Encoding

The *generator matrix* $G \in \mathbb{Z}_2^{7\times 4}$ has columns that form a basis for $\ker(H)$:

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix}.$$

Encoding a 4-bit message $m \in \mathbb{Z}_2^4$ produces the codeword $c = Gm \in \mathcal{C}$. One verifies $HG = 0 \pmod 2$, confirming every column of G lies in $\ker(H)$.

Remark 7.1 (Systematic form)**.** The top 4×4 block of G is I_4, so the first four bits of the codeword $c = Gm$ are the original message m unchanged. The bottom three bits are *parity bits* that carry redundancy.

Error detection and correction via the syndrome

Suppose the received word is $r = c + e \in \mathbb{Z}_2^7$, where $c \in \mathcal{C}$ and e is an error vector. Compute the *syndrome*:

$$s = Hr = H(c + e) = Hc + He = 0 + He = He.$$

- $s = 0$: no error detected ($r \in \ker H$).
- $s \neq 0$: $s = He$ is a linear combination of columns of H weighted by e.

For a *single-bit error* $e = e_k$ (the k-th standard basis vector), $s = He = H_k$, the k-th column of H. Since the columns of H are the numbers $1, \ldots, 7$ in binary, the syndrome s is exactly the binary representation of the error position k.

Theorem 7.1 (Single-bit correction)**.** *If at most one bit is flipped, the syndrome $s = Hr$ is either 0 (no error) or the binary index of the corrupted bit. Correcting: flip bit k where $k = \mathrm{bin2dec}(s)$.*

Proof. The columns of H are all 7 nonzero elements of $\mathbb{Z}_2^3$, all distinct. For a single-bit error $e = e_k$, the syndrome $He = H_k$ uniquely identifies k. For $e = 0$ the syndrome is 0. No two single-bit errors produce the same syndrome, so correction is unambiguous. □

Remark 7.2 (Why 7 bits?)**.** With r parity-check bits one can index $2^r - 1$ positions. For $r = 3$: $2^3 - 1 = 7$, which accounts for the 3 parity positions and leaves $7 - 3 = 4$ data bits. This is the most efficient possible single-error-correcting code for 4 data bits.

Python: encoding, flipping, correcting

```
import numpy as np

# All arithmetic mod 2
def z2(A): return A % 2

H = np.array([[0,0,0,1,1,1,1],
```

```
             [0,1,1,0,0,1,1],
             [1,0,1,0,1,0,1]], dtype=int)

G = np.array([[1,0,0,0],
             [0,1,0,0],
             [0,0,1,0],
             [0,0,0,1],
             [1,1,0,1],
             [1,0,1,1],
             [0,1,1,1]], dtype=int)

# Verify HG = 0 (mod 2)
print("HG mod 2:\n", z2(H @ G)) # should be all zeros

# Encode message m = [1,0,1,1]
m = np.array([1,0,1,1])
c = z2(G @ m)
print("\nmessage: ", m)
print("codeword: ", c)
print("syndrome of codeword:", z2(H @ c)) # 0: valid codeword

# Introduce a single-bit error at position 5 (0-indexed: bit 4)
r = c.copy()
r[4] ^= 1
print("\nreceived (with error at bit 4):", r)

# Compute syndrome
s = z2(H @ r)
print("syndrome:", s) # binary representation of error position

# Decode syndrome to error position (1-indexed)
error_pos = int("".join(map(str, s)), 2)
print("error at position:", error_pos) # 5 (1-indexed)

# Correct
r_fixed = r.copy()
r_fixed[error_pos - 1] ^= 1
print("corrected:", r_fixed)
print("matches original codeword:", np.array_equal(r_fixed, c)) # True
```

8 Polynomial Regression

Setup

Recall that $\mathcal{P}_d = \{a_0 + a_1 x + a_2 x^2 + \cdots + a_d x^d : a_i \in \mathbb{R}\} \cong \mathbb{R}^{d+1}$ (the isomorphism $p \leftrightarrow (a_0, \ldots, a_d)^T$ was established in §1; we now use it concretely).

Problem. Given a data cloud (x_i, y_i), $i = 1, \ldots, N$, find a polynomial $p \in \mathcal{P}_d$ that "models" the data.

Ideal requirement: $p(x_i) = y_i$ for all i. **Obstruction:** this may not be feasible. There are two qualitatively different failure modes:

1. **Underdetermined model.** d is too small relative to N (e.g. $d = 3$, $N = 10^3$): we have $d+1$ unknowns but N equations, and the system is generically overdetermined.

2. **Inconsistent data.** Two points share an x-value but have different y-values ($x_1 \approx x_2$, $y_1 \not\approx y_2$): no polynomial of any degree can fit both exactly.

Remedy: satisfy the equalities *approximately*, $p(x_i) \approx y_i$. This gives the system:

$$\begin{cases} a_0 + a_1 x_1 + a_2 x_1^2 + \cdots + a_d x_1^d = y_1 \\ \quad\vdots \\ a_0 + a_1 x_N + a_2 x_N^2 + \cdots + a_d x_N^d = y_N \end{cases} \tag{2}$$

This is N equations in $d+1$ unknowns — almost a matrix equation $X\beta = y$, which we make precise on the next page.

Python: why exact fitting fails when $N \gg d$

```
import numpy as np

rng = np.random.default_rng(0)
N, d = 30, 3
x = np.linspace(-1, 1, N)
y = 2*x**2 - x + 1 + 0.3*rng.standard_normal(N) # noisy quadratic

# Exact system: N equations, d+1 unknowns
print(f"{N} equations, {d+1} unknowns -> overdetermined, generally no exact solution")

# Least-squares fit (exact solution replaced by best approximation)
X = np.column_stack([x**k for k in range(d+1)]) # Vandermonde matrix
beta, res, rank, sv = np.linalg.lstsq(X, y, rcond=None)
print(f"fitted coefficients: {beta}")
print(f"residual ||Xbeta - y||: {np.linalg.norm(X @ beta - y):.4f}")
```

Matrix form and normal equations

Encode the unknowns, data, and observations as vectors and a matrix:

$$\begin{aligned} \beta &= (a_0, a_1, \ldots, a_d)^T \in \mathbb{R}^{d+1} && \text{(polynomial coefficients as a vector)} \\ X &= \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ \vdots & & & & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^d \end{pmatrix} \in \mathbb{R}^{N\times(d+1)} && \text{(feature / Vandermonde matrix)} \\ y &= (y_1, \ldots, y_N)^T \in \mathbb{R}^N && \text{(true values)} \end{aligned}$$

Remark 8.1 (Vandermonde matrix)**.** X is a *Vandermonde matrix*: each row i evaluates the monomials $1, x, x^2, \ldots, x^d$ at x_i. The same structure appears in polynomial interpolation, discrete Fourier transforms, and spectral methods.

The exact fitting requirement $p(x_i) = y_i$ for all i is:

$$X\beta = y, \qquad X \in \mathbb{R}^{N\times(d+1)},\ \beta \in \mathbb{R}^{d+1},\ y \in \mathbb{R}^N. \tag{$*$}$$

In most cases $(*)$ has no solution ($y \notin \operatorname{im} X$).

Instead, solve the **normal equations**:

$$X^T X \beta = X^T y. \tag{$**$}$$

$X^T X$ is $(d+1) \times (d+1)$ and likely invertible, giving $\beta = (X^T X)^{-1} X^T y$.

Remark 8.2 (Why the normal equations work — forward pointer)**.** Equation $(**)$ is not a trick: it is the condition $y - X\beta \perp \operatorname{im} X$, i.e. the residual is orthogonal to the column space of X. The geometric interpretation — projecting y onto $\operatorname{im} X$ — will be made precise once we have inner products and projections (§3).

Remark 8.3 (Computational size reduction)**.** X is $N \times (d+1)$ with $N \gg d$: huge. But $X^T X$ is only $(d+1) \times (d+1)$, independent of N. Solving $(**)$ instead of $(*)$ is not only geometrically justified — it is why least-squares regression is computationally tractable for large datasets.

Python: Vandermonde matrix and normal equations

```
import numpy as np

rng = np.random.default_rng(0)
N, d = 30, 3
x = np.linspace(-1, 1, N)
y = 2*x**2 - x + 1 + 0.3*rng.standard_normal(N)

# Build Vandermonde (feature) matrix explicitly
X = np.column_stack([x**k for k in range(d + 1)]) # shape (N, d+1)
print("X shape:", X.shape) # (30, 4)
print("X^T X shape:", (X.T @ X).shape) # (4, 4) -- much smaller

# Solve normal equations
beta_normal = np.linalg.solve(X.T @ X, X.T @ y)

# Compare with numpy's least-squares solver
beta_lstsq, _, _, _ = np.linalg.lstsq(X, y, rcond=None)

print("normal equations:", beta_normal)
print("lstsq: ", beta_lstsq) # should match
```

From polynomial regression to inner products

To minimise $\|X\beta - y\|$ we need a notion of distance: $\|a - b\|$ measures how close a is to b. The norm $\|X\beta - y\|$ we want to minimise is $\sqrt{\langle X\beta - y,\, X\beta - y\rangle}$ under the standard inner product on $\mathbb{R}^N$ — the abstract definition below directly powers this computation.

Thus, we don't need merely vector spaces; we need vector spaces **with inner products and norms**.

9 Inner Products and Norms (§2.1)

Definition and axioms

Toy example. $\mathbb{R}^2$ with the usual dot product: $v = (a, b)$, $\|v\|^2 = a^2 + b^2 = v \cdot v$.

Definition 9.1 (Inner product)**.** Let V be a vector space over $F = \mathbb{R}$ or $\mathbb{C}$. An *inner product* on V is a function $\langle \cdot, \cdot \rangle : V \times V \to F$ satisfying:

1. **Positivity** (positive definiteness): $\langle v, v \rangle \geq 0 \; \forall \, v \in V$, with $\langle v, v \rangle = 0 \Leftrightarrow v = 0$.
2. **Linearity in the first argument**:

$$\begin{aligned} \langle u + v,\, w \rangle &= \langle u, w \rangle + \langle v, w \rangle && \forall \, u, v, w \in V, \\ \langle \lambda u,\, v \rangle &= \lambda \langle u, v \rangle && \forall \, \lambda \in F,\; u, v \in V. \end{aligned}$$

3. **Symmetry**: $\langle u, v \rangle = \langle v, u \rangle$ if $F = \mathbb{R}$; $\quad \langle u, v \rangle = \overline{\langle v, u \rangle}$ if $F = \mathbb{C}$ (Hermitian symmetry).

Remark 9.1 (Linearity convention)**.** Axiom 2 places linearity in the *first* argument. Physicists (Dirac bra-ket notation) conventionally put linearity in the second; mathematicians in the first. Both conventions appear in standard references — always check which one is in use.

Remark 9.2 (Conjugate-linearity in the second argument)**.** Axioms 2 and 3 together imply:

$$\langle u,\, \lambda v \rangle = \bar{\lambda} \, \langle u, v \rangle \quad \forall \, \lambda \in F.$$

For $F = \mathbb{R}$ this is ordinary linearity; for $F = \mathbb{C}$ it is *conjugate-linearity* (sesquilinearity). The inner product is *bilinear* only over $\mathbb{R}$.

Python: inner product axioms

```
import numpy as np

def inner(u, v, W=None):
    """Standard (W=None) or weighted inner product u^T W v on R^n."""
    if W is None:
        return float(u @ v)
    return float(u @ W @ v)

u = np.array([1., 2., 3.])
v = np.array([4., 5., 6.])
W = np.diag([1., 2., 3.]) # positive definite weight matrix

print("standard <u,v>:", inner(u, v))
print("weighted <u,v>:", inner(u, v, W))

# Verify axioms for weighted inner product
print("positivity <u,u> >= 0:", inner(u, u, W) >= 0) # True
print("symmetry <u,v>==<v,u>:", np.isclose(inner(u, v, W), inner(v, u, W))) # True
lam = 3.0
print("linearity:", np.isclose(inner(lam*u, v, W), lam * inner(u, v, W))) # True
```

Examples of inner products

Example 9.1 (Euclidean inner product on $\mathbb{R}^n$)**.**

$$\langle (x_1, \ldots, x_n)^T,\, (y_1, \ldots, y_n)^T \rangle = \sum_{i=1}^{n} x_i y_i.$$

Example 9.2 (Weighted Euclidean inner product)**.** If $c_1, \ldots, c_n > 0$, then $\langle x, y\rangle_c = \sum_{i=1}^n c_i x_i y_i$ defines an inner product on $\mathbb{R}^n$.

Remark 9.3 (Connection to weighted regression)**.** This is the inner product underlying *weighted least squares*: setting c_i to the confidence in data point i assigns higher weight to more reliable observations. The normal equations $X^T Xb = X^T y$ become $X^T CXb = X^T Cy$ with diagonal weight matrix $C = \mathrm{diag}(c_1, \ldots, c_N)$.

Example 9.3 (Inner products on $\mathcal{P}_n$)**.**

- $\langle p, q\rangle = p(0)q(0) + \displaystyle\int_{-1}^{1} p'(x)q'(x)\,dx.$
- $\langle p, q\rangle = \displaystyle\int_0^\infty p(x)q(x)e^{-x}\,dx \quad$ (L^2 with exponential weight).
- Via the isomorphism $\phi(p) = (a_0, a_1, \ldots, a_n)^T$: $\langle p, q\rangle = \phi(p) \cdot \phi(q)$ (standard dot product on $\mathbb{R}^{n+1}$).

Positivity check for the first variant. $\langle p, p\rangle = p(0)^2 + \int_{-1}^1 (p')^2\,dx = 0$ requires $p(0) = 0$ and $p'(x) = 0$ for all $x \in [-1, 1]$, which forces $p = 0$. So the definiteness axiom holds. □

Definition 9.2 (Inner product space)**.** An *inner product space* is a vector space V equipped with an inner product.

Proposition 9.1 (3.1.1)**.**

(a) For any fixed $u \in V$, the map $f(v) = \langle v, u\rangle : V \to F$ is a linear functional.

(b) $\langle 0, v\rangle = 0$ and $\langle v, 0\rangle = 0$ for all $v \in V$.

Proof. (a) follows directly from axiom 2 (linearity in the first argument).

(b) $\langle 0, v\rangle = \langle 0 \cdot u, v\rangle = 0 \cdot \langle u, v\rangle = 0$ for any $u \in V$. For the second: $\langle v, 0\rangle = \overline{\langle 0, v\rangle} = \bar{0} = 0$ by symmetry/Hermitian symmetry. □

Python: polynomial inner products

```
import numpy as np
from scipy.integrate import quad

# Two polynomials: p = 1 + 2x + x^2, q = x - x^2
# Represented as coefficient vectors (a0, a1, a2)
p_c = np.array([1., 2., 1.])
q_c = np.array([0., 1., -1.])

# Inner product via phi isomorphism (coefficient dot product)
print("coeff dot <p,q>:", p_c @ q_c) # = 0*1 + 1*2 + (-1)*1 = 1

# L^2 weighted inner product: integral_0^inf p(x)q(x)e^{-x} dx
p = np.poly1d(p_c[::-1]) # np.poly1d uses highest-degree first
q = np.poly1d(q_c[::-1])
val, _ = quad(lambda x: p(x) * q(x) * np.exp(-x), 0, np.inf)
print("L2 weighted <p,q>:", round(val, 6)) # different value: different inner product
```

Derived properties, norm, orthogonality

The following properties are consequences of the axioms (not additional assumptions):

(c) $\langle u, v+w\rangle = \langle u, v\rangle + \langle u, w\rangle, \quad \forall\, u, v, w \in V.$

(d) $\langle u, \lambda v\rangle = \bar{\lambda}\,\langle u, v\rangle, \quad \forall\, \lambda \in F,\ u, v \in V.$

(Both follow from axioms 2 and 3: apply linearity in the first argument to $\langle v+w, u\rangle$ and $\langle \lambda v, u\rangle$, then use symmetry.)

Definition 9.3 (Norm induced by an inner product)**.**

$$\|v\|^2 = \langle v, v\rangle, \qquad \|v\| = \sqrt{\langle v, v\rangle}.$$

Proposition 9.2 (3.1.2)**.** *For any $v \in V$ and $\lambda \in F$:*

(a) $\|v\| = 0 \iff v = 0.$

(b) $\|\lambda v\| = |\lambda|\,\|v\|.$

Proof. (a) $\|v\| = 0 \Leftrightarrow \langle v, v\rangle = 0 \Leftrightarrow v = 0$ by the positivity axiom.

(b) $\|\lambda v\|^2 = \langle \lambda v, \lambda v\rangle = \lambda\,\langle v, \lambda v\rangle = \lambda\bar{\lambda}\,\langle v, v\rangle = |\lambda|^2\|v\|^2$. Taking square roots gives $\|\lambda v\| = |\lambda|\,\|v\|$. □

Definition 9.4 (Orthogonality)**.** $u, v \in V$ are *orthogonal*, written $u \perp v$, if $\langle u, v\rangle = 0$.

Example 9.4. 0 is orthogonal to every $v \in V$ (by Prop 3.1.1(b)). It is the *only* vector orthogonal to itself: if $\langle v, v\rangle = 0$ then $v = 0$ by positivity.

Theorem 9.1 (3.1.3 — Pythagorean theorem)**.** *If $u \perp v$, then $\|u+v\|^2 = \|u\|^2 + \|v\|^2$.*

Proof.

$$\|u+v\|^2 = \langle u+v,\, u+v\rangle = \|u\|^2 + \langle u, v\rangle + \langle v, u\rangle + \|v\|^2 = \|u\|^2 + 0 + 0 + \|v\|^2. \qquad \square$$

Definition 9.5 (Orthogonal projection onto $v \neq 0$)**.**

$$\mathrm{proj}_v(u) = \frac{\langle u, v\rangle}{\|v\|^2}\, v.$$

Remark 9.4 (Why this formula)**.** $\mathrm{proj}_v(u)$ is the unique scalar multiple of v closest to u. The residual $u - \mathrm{proj}_v(u)$ is orthogonal to v:

$$\left\langle u - \tfrac{\langle u,v\rangle}{\|v\|^2} v,\ v \right\rangle = \langle u, v\rangle - \tfrac{\langle u,v\rangle}{\|v\|^2}\|v\|^2 = 0.$$

This residual-orthogonality is the same condition as the normal equations $X^T X b = X^T y$: the projection of y onto $\operatorname{im} X$ makes the residual $y - Xb$ orthogonal to every column of X.

Python: projection and orthogonality of residual

```
import numpy as np

u = np.array([3., 2.])
v = np.array([1., 0.]) # project onto x-axis

proj = (np.dot(u, v) / np.dot(v, v)) * v
residual = u - proj

print("proj_v(u):", proj) # [3. 0.]
print("residual:", residual) # [0. 2.]
print("residual perp v:", np.isclose(np.dot(residual, v), 0)) # True
print("Pythagorean check:", np.isclose(
    np.dot(u, u),
    np.dot(proj, proj) + np.dot(residual, residual))) # True
```

Proposition 9.3 (3.1.4)**.** *If $v \neq 0$, then $\mathrm{proj}_v(u)$ and $w = u - \mathrm{proj}_v(u)$ are orthogonal for all $u \in V$.*

Proof.

$$\big\langle u - \mathrm{proj}_v(u),\ v \big\rangle = \langle u, v\rangle - \frac{\langle u, v\rangle}{\|v\|^2}\,\langle v, v\rangle = \langle u, v\rangle - \langle u, v\rangle = 0. \qquad \square$$

Theorem 9.2 (3.1.5 — Cauchy–Schwarz)**.** *For all $u, v \in V$:*

$$|\langle u, v\rangle| \leq \|u\|\,\|v\|,$$

with equality if and only if $u = \lambda v$ for some $\lambda \in F$.

Proof. If $v = 0$ both sides are 0. Assume $v \neq 0$. For any $\lambda \in \mathbb{R}$, positivity gives

$$0 \leq \|u - \lambda v\|^2 = \|u\|^2 - 2\lambda\,\mathrm{Re}\langle u, v\rangle + \lambda^2\|v\|^2.$$

This is a quadratic in λ that is always ≥ 0, so its discriminant is non-positive:

$$4(\mathrm{Re}\langle u, v\rangle)^2 - 4\|u\|^2\|v\|^2 \leq 0 \implies |\langle u, v\rangle|^2 \leq \|u\|^2\|v\|^2.$$

Equality holds iff the quadratic has a real root, i.e. $u = \lambda v$. $\square$

Theorem 9.3 (3.1.6 — Triangle Inequality)**.** *For all $u, v \in V$: $\|u + v\| \leq \|u\| + \|v\|$, with equality iff $u = \lambda v$ for some $\lambda > 0$.*

Proof.

$$\|u+v\|^2 = \|u\|^2 + 2\,\mathrm{Re}\langle u, v\rangle + \|v\|^2 \leq \|u\|^2 + 2|\langle u, v\rangle| + \|v\|^2 \leq \|u\|^2 + 2\|u\|\|v\| + \|v\|^2 = \big(\|u\| + \|v\|\big)^2,$$

where the last inequality is Cauchy–Schwarz. Take square roots. $\square$

Exercise 9.1 (Parallelogram equality)**.** Show that for all $u, v \in V$:

$$\|u + v\|^2 + \|u - v\|^2 = 2\|u\|^2 + 2\|v\|^2.$$

Remark 9.5 (Parallelogram law characterises inner product norms)**.** The Jordan–von Neumann theorem states that a normed space has its norm induced by an inner product *if and only if* the parallelogram equality holds. Consequently, L^p norms for $p \neq 2$ do *not* come from inner products — a fact that matters in functional analysis, robust regression, and any setting where $\|\cdot\|_p$ with $p \neq 2$ appears.

Definition 9.6 (Orthogonal complement)**.** If $U \leq V$, the *orthogonal complement* of U is

$$U^\perp = \{v \in V : \langle v, u\rangle = 0 \; \forall\, u \in U\}.$$

Back to polynomial regression: why the normal equations work

Let $U = \operatorname{im} X \leq \mathbb{R}^N$ be the column space of the feature matrix. We want $\hat\beta$ minimising $\|y - X\beta\|$.

Geometric argument. Project y onto U: the projection $\hat y = \operatorname{proj}_U(y)$ always exists (inner product space, finite dimension) and is unique. By definition of projection, $y - \hat y \perp U$, i.e. $\langle y - \hat y, Xv\rangle = 0$ for all $v \in \mathbb{R}^{d+1}$. Write $\hat y = X\hat\beta$; then

$$\langle y - X\hat\beta,\, Xv\rangle = 0 \;\; \forall\, v \quad\Longrightarrow\quad X^T(y - X\hat\beta) = 0 \quad\Longrightarrow\quad X^TX\hat\beta = X^Ty.$$

The minimiser satisfies the normal equations. $\hat y$ is unique; $\hat\beta$ need not be if $\ker X \neq \{0\}$.

Algebraic argument (normal equations always have a solution). It is not obvious that $X^Ty \in \operatorname{im}(X^TX)$. The following lemma shows it.

Proposition 9.4 ($\ker(X^TX) = \ker(X)$)**.** *For any $X \in \mathbb{R}^{N\times(d+1)}$:*

$$\ker(X^TX) = \ker(X).$$

Proof. $\ker(X) \subseteq \ker(X^TX)$: if $Xv = 0$ then $X^TXv = 0$.
$\ker(X^TX) \subseteq \ker(X)$: if $X^TXv = 0$, then $\|Xv\|^2 = \langle Xv, Xv\rangle = v^TX^TXv = 0$, so $Xv = 0$. □

Corollary 9.1. $\operatorname{im}(X^TX) = \operatorname{im}(X^T)$*, so $X^Ty \in \operatorname{im}(X^TX)$ for every y. Hence the normal equations always have a solution.*

Proof. By the Im-Ker Theorem, $\operatorname{rk}(X^TX) = \operatorname{rk}(X) = \operatorname{rk}(X^T)$. Since $\operatorname{im}(X^TX) \subseteq \operatorname{im}(X^T)$ and both have the same dimension, they are equal. Finally $X^Ty = X^T \cdot y \in \operatorname{im}(X^T)$ trivially. □

The complete chain of equivalences:

$$\|y - X\beta\| \to \min \iff y - X\beta \perp \operatorname{im} X \iff X^TX\beta = X^Ty.$$

Definition 9.7 (Hat matrix / projection matrix)**.** The *hat matrix* $H = X(X^TX)^{-1}X^T$ (when X has full column rank) satisfies $\hat y = Hy$ and:

$$H^2 = H \quad \text{(idempotent)}, \qquad H^T = H \quad \text{(symmetric)}.$$

These two properties characterise orthogonal projections.

Python: hat matrix and projection

```
import numpy as np

rng = np.random.default_rng(0)
N, d = 30, 3
x = np.linspace(-1, 1, N)
y = 2*x**2 - x + 1 + 0.3*rng.standard_normal(N)
X = np.column_stack([x**k for k in range(d + 1)])

# Hat matrix
H = X @ np.linalg.solve(X.T @ X, X.T)
y_hat = H @ y # projection of y onto im X
resid = y - y_hat

print("H^2 = H:", np.allclose(H @ H, H)) # True
print("H^T = H:", np.allclose(H.T, H)) # True
print("resid perp im X:", np.allclose(X.T @ resid, 0)) # True (normal eqs)
print("||resid||:", np.linalg.norm(resid).round(4))
```

Completing the normal equations derivation

Lemma 9.1. *For a fixed $a \in \mathbb{R}^n$: if $\langle a, b\rangle = 0$ for all $b \in \mathbb{R}^n$, then $a = 0$.*

Proof. Take $b = a$: then $\langle a, a\rangle = 0$, so $a = 0$ by positivity. □

Applying Lemma 9.1: from $\langle X^T X\beta - X^T y,\, v\rangle = 0$ for all $v \in \mathbb{R}^{d+1}$ we conclude $X^T X\beta - X^T y = 0$, i.e. $X^T X\beta = X^T y$.

Question: in most cases $X^T X$ has an inverse. Why?

Proposition 9.5 (Invertibility of $X^T X$)**.** *$X^T X$ is invertible if and only if $\mathrm{rk}(X) = d+1$ (columns of X are linearly independent).*

Proof. $X^T X$ is invertible iff $\ker(X^T X) = \{0\}$ iff $\ker(X) = \{0\}$ (by the proposition on page 28) iff $\mathrm{rk}(X) = d + 1$. □

Remark 9.6 (When is this satisfied?)**.** The columns of X are linearly independent when the nodes $x_1, \ldots, x_N$ are *distinct* and $N \geq d + 1$. A Vandermonde matrix with distinct nodes is always full rank (its determinant is $\prod_{i<j}(x_j - x_i) \neq 0$). "In most cases" means precisely this: the degenerate cases are repeated x-values or fewer data points than parameters.

Summary. The full chain:

$$\|y - X\beta\| \to \min \iff y - X\beta \perp \operatorname{im} X \iff X^T X\beta = X^T y \xRightarrow{X^T X \text{ invertible}} \beta^* = (X^T X)^{-1} X^T y.$$

This closes the loop opened on page 23.

Python: rank of $X^T X$ for good vs. degenerate data

```
import numpy as np

d = 3 # degree

# Generic case: distinct nodes, N >> d+1
x_good = np.linspace(-1, 1, 30)
X_good = np.column_stack([x_good**k for k in range(d + 1)])
print("rank(X^TX) good:", np.linalg.matrix_rank(X_good.T @ X_good)) # 4

# Degenerate: all nodes the same
x_bad = np.zeros(30)
X_bad = np.column_stack([x_bad**k for k in range(d + 1)])
print("rank(X^TX) bad:", np.linalg.matrix_rank(X_bad.T @ X_bad)) # 1

# Degenerate: fewer points than parameters
x_few = np.array([-1., 0., 1.]) # only 3 points, d+1 = 4 unknowns
X_few = np.column_stack([x_few**k for k in range(d + 1)])
print("rank(X^TX) few:", np.linalg.matrix_rank(X_few.T @ X_few)) # 3 < 4
```

10 Im-Ker Theorem: Full Proof

Terminology and statement

Going back to Theorem 1.6 (Im-Ker / Rank-Nullity), we now give the full proof in the abstract linear map setting.

Recall the terminology:

$$\begin{aligned} \operatorname{im} A &= \operatorname{col}(A) = \text{column space of } A, \\ \operatorname{im} A^T &= \operatorname{row}(A) = \text{row space of } A, \\ \operatorname{null} A &= \dim \ker A \quad (\text{“nullity” of } A), \\ \ker A &= \text{“null-space” of } A. \end{aligned}$$

Remark 10.1 (Row rank equals column rank)**.** The equality $\operatorname{rk} A = \dim \operatorname{im} A \stackrel{?}{=} \dim \operatorname{im} A^T$ is a genuine theorem, not obvious. It follows from combining the Im-Ker Theorem applied to both A and A^T with the identity $\operatorname{rk}(A^T A) = \operatorname{rk}(A)$ proved on page 28 (via $\ker(A^T A) = \ker(A)$).

Theorem 10.1 (1.6, matrix form)**.** *For any $A \in \mathbb{R}^{m \times n}$:* $\operatorname{rk} A + \operatorname{null} A = n$.

Theorem 10.2 (1.6, linear transform form — Im-Ker Theorem)**.** *For any $L \in \mathcal{L}(V, W)$:* $\dim \operatorname{im} L + \dim \ker L = \dim V$.

Proof. **Proof strategy:** split $V = \ker L \oplus C$ (complement), then show $\{L(u_1), \ldots, L(u_r)\}$ is a basis for $\operatorname{im} L$.

Let $V_1 = \ker L \leq V$ and $\dim V_1 = k \leq n = \dim V$. Pick a basis $\{b_1, \ldots, b_k\}$ for V_1. By the Steinitz extension (pages 18–19), extend to a basis of V by adjoining $\{u_1, \ldots, u_r\}$ with $k + r = n$:

$$V = \langle b_1 \rangle \oplus \cdots \oplus \langle b_k \rangle \oplus \langle u_1 \rangle \oplus \cdots \oplus \langle u_r \rangle.$$

Every $v \in V$ writes uniquely as $v = \sum_{i=1}^{k} \lambda_i b_i + \sum_{j=1}^{r} \mu_j u_j$. *(Proof continues on page 31.)* $\square$

Python: row rank equals column rank

```
import numpy as np
rng = np.random.default_rng(0)

A = rng.standard_normal((5, 8))
print("col rank (rank of A): ", np.linalg.matrix_rank(A)) # e.g. 5
print("row rank (rank of A^T):", np.linalg.matrix_rank(A.T)) # same

# True for any matrix -- a non-trivial theorem
B = rng.standard_normal((7, 3))
print("col rank:", np.linalg.matrix_rank(B)) # 3
print("row rank:", np.linalg.matrix_rank(B.T)) # 3
```

Proof of Im-Ker Theorem (conclusion)

Proof (continued from page 30). Let $V_1 = \ker L$ and $V_2 = \operatorname{span}\{u_1, \dots, u_r\}$, so that $V = V_1 \oplus V_2$ (direct sum from the basis construction).

Step 1: $\operatorname{im} L = \operatorname{span}\{L(u_1), \dots, L(u_r)\}$. Since $b_i \in \ker L$, applying L to the decomposition of v gives

$$L(v) = \sum_{i=1}^{k} \lambda_i \underbrace{L(b_i)}_{=0} + \sum_{j=1}^{r} \mu_j L(u_j) = \sum_{j=1}^{r} \mu_j L(u_j).$$

So every $L(v) \in \operatorname{span}\{L(u_1), \dots, L(u_r)\}$, giving $\operatorname{im} L \subseteq \operatorname{span}\{L(u_1), \dots, L(u_r)\}$; the reverse inclusion is immediate.

Step 2: $\{L(u_1), \dots, L(u_r)\}$ is linearly independent. Suppose $\sum_{j=1}^{r} \mu_j L(u_j) = 0$. Then $L\big(\sum_j \mu_j u_j\big) = 0$, so $u := \sum_j \mu_j u_j \in \ker L = V_1$. But also $u \in V_2$ by construction. Since $V = V_1 \oplus V_2$ we have $V_1 \cap V_2 = \{0\}$, so $u = 0$. The u_j are linearly independent, so $\mu_j = 0$ for all j.

Conclusion. Steps 1 and 2 together show $\{L(u_1), \dots, L(u_r)\}$ is a *basis* for $\operatorname{im} L$, giving $\dim \operatorname{im} L = r$. With $\dim \ker L = k$ and $k + r = n = \dim V$:

$$\dim \operatorname{im} L + \dim \ker L = r + k = n = \dim V. \qquad \square$$

Remark 10.2 (Connection to column reduction)**.** The proof constructs exactly what the algorithm of pages 16–17 computes: $V_2 = \operatorname{span}\{u_1, \dots, u_r\}$ corresponds to the *pivot columns* (a basis for $\operatorname{im} A$), while the *free columns* (zero top-block columns) generate $\ker A$. The abstract decomposition $V = V_1 \oplus V_2$ is what the column reduction algorithm realises concretely.

Remark 10.3 (First isomorphism theorem)**.** The proof shows that the restricted map $L|_{V_2} : V_2 \to \operatorname{im} L$ is an isomorphism. Equivalently, L induces a well-defined bijection on the *quotient space* $V/\ker L \xrightarrow{\sim} \operatorname{im} L$, which is the **first isomorphism theorem** for vector spaces. The dimension count $\dim \operatorname{im} L = \dim V - \dim \ker L$ is its immediate corollary.

Row space, orthogonal complements, and the four subspaces

Recall that Ax is a linear combination of the columns of A, but also:

$$Ax = \begin{pmatrix} a_1^T x \\ \vdots \\ a_m^T x \end{pmatrix}, \qquad a_i = i\text{-th row of } A = i\text{-th column of } A^T.$$

Hence

$$Ax = 0 \iff a_i^T x = 0 \ \forall i \iff x \perp \langle a_1, \dots, a_m \rangle = \mathrm{row}(A).$$

So $\ker A = \big(\mathrm{row}(A)\big)^\perp$ inside $\mathbb{R}^m$.

Proposition 10.1 (3.1.7, Homework)**.** *$U^\perp \leq V$.*

Proof. $\langle 0, u\rangle = 0$ for all u, so $0 \in U^\perp$. If $v, w \in U^\perp$ and $\lambda \in F$, then for all $u \in U$: $\langle v + w, u\rangle = \langle v, u\rangle + \langle w, u\rangle = 0$ and $\langle \lambda v, u\rangle = \lambda\langle v, u\rangle = 0$. So $U^\perp$ is closed under addition and scaling. □

Proposition 10.2 (3.1.8, Homework)**.** *$V = U \oplus U^\perp$.*

Proof sketch. $U \cap U^\perp = \{0\}$*:* if $v \in U \cap U^\perp$ then $\langle v, v\rangle = 0$, so $v = 0$ by positivity.

$V = U + U^\perp$*:* let $\{e_1, \dots, e_k\}$ be an orthonormal basis for U (existence via Gram–Schmidt, §3.2). For any $v \in V$, set $u = \sum_i \langle v, e_i\rangle e_i \in U$. Then $v - u \perp e_j$ for each j (direct computation), so $v - u \in U^\perp$ and $v = u + (v - u)$. □

Taking $U = \mathrm{row}(A) \leq \mathbb{R}^m$:

$$\mathbb{R}^m = \mathrm{row}(A) \oplus \ker(A).$$

Hence $m = \dim \mathrm{row}(A) + \dim \ker(A) = \dim \mathrm{row}(A) + \mathrm{null}(A)$. By Thm 1.6, $m = \mathrm{rk}(A) + \mathrm{null}(A)$, giving:

$$\boxed{\mathrm{rk}(A) = \dim \mathrm{row}(A).} \qquad \text{(row rank = column rank)}$$

Remark 10.4 (Strang's four fundamental subspaces)**.** Applying $V = U \oplus U^\perp$ to both A and A^T gives the full picture:

$$\mathbb{R}^m = \mathrm{row}(A) \oplus \ker(A),$$
$$\mathbb{R}^n = \mathrm{col}(A) \oplus \ker(A^T).$$

The four subspaces — column space, row space, null space, left null space — come in two orthogonal pairs, one pair in $\mathbb{R}^m$, one in $\mathbb{R}^n$. Their dimensions are fixed by $\mathrm{rk}(A)$ alone.

Remark 10.5 ($V = U \oplus U^\perp$ as a recurring theme)**.** This decomposition is the structural engine behind Gram–Schmidt (building orthonormal bases), orthogonal projections, the normal equations, and eventually SVD. The fact that every subspace has an orthogonal complement — and that the two together fill the whole space — is what makes inner product spaces so much richer than plain vector spaces.

Python: the four fundamental subspaces are mutually orthogonal

```
import numpy as np
from scipy.linalg import null_space, orth

rng = np.random.default_rng(0)
A = rng.standard_normal((4, 6)) # 4x6, rank 4 generically

row_A = orth(A.T) # basis for row(A) in R^6
ker_A = null_space(A) # basis for ker(A) in R^6
col_A = orth(A) # basis for col(A) in R^4
lnul_A = null_space(A.T) # basis for ker(A^T) in R^4
```

```
print("row(A) perp ker(A):", np.allclose(row_A.T @ ker_A, 0)) # True
print("col(A) perp ker(A):", np.allclose(col_A.T @ lnul_A, 0)) # True

# Dimension check
print(f"R^6: dim row={row_A.shape[1]}, dim ker={ker_A.shape[1]}, sum={row_A.shape[1]+
    ker_A.shape[1]}") # 4+2=6
print(f"R^4: dim col={col_A.shape[1]}, dim lnul={lnul_A.shape[1]}, sum={col_A.shape[1]+
    lnul_A.shape[1]}") # 4+0=4
```

Theorem 10.3 (3.1.9 — Row rank equals column rank)**.** *For any $A \in \mathbb{R}^{m\times n}$:*

$$\textit{column-rank}(A) = \textit{row-rank}(A) = \operatorname{rk} A,$$

where column-rank(A) = max # of linearly independent columns, and row-rank(A) = max # of linearly independent rows.

Proof. By page 32: $\mathbb{R}^m = \operatorname{row}(A) \oplus \ker(A)$, so $\dim \operatorname{row}(A) = m - \operatorname{null}(A)$. By Thm 1.6: $\operatorname{rk}(A) = m - \operatorname{null}(A)$. Hence row-rank$(A) = \operatorname{rk}(A) = \dim \operatorname{im}(A) =$ column-rank(A). □

11 Structural Properties of Linear Transformations (§4)

§4.1 Change of basis

Main question. Let $L : V \to W$ be a linear transform, $\dim V = n$, $\dim W = m$. Can we "simplify" the action of L by choosing a good basis?

Remark 11.1 (What "simpler" means)**.** The target normal forms, in increasing generality:

- **Diagonal** — achievable when L has n linearly independent eigenvectors (diagonalisable maps, §4.2).
- **Upper triangular** — always achievable over $\mathbb{C}$ (Schur decomposition).
- **Jordan normal form** — the finest canonical form over $\mathbb{C}$, grouping generalised eigenvectors into Jordan blocks.

For now we focus on the case $V = W = \mathbb{R}^n$ and ask: in what basis is the matrix of L simplest?

Let A be the matrix of $L : x \mapsto Ax$ in the *standard* basis of $\mathbb{R}^n$. Let $B = [b_1 \mid \cdots \mid b_n]$ be another basis matrix (invertible, columns are the new basis vectors).

Change-of-basis formula. A vector with standard coordinates x has B-coordinates $x' = B^{-1}x$. Applying L:

$$x \overset{L}{\mapsto} Ax \implies x' \overset{L_B}{\longmapsto} B^{-1}AB\,x'.$$

So the matrix of L in basis B is $\boxed{A_B = B^{-1}AB}$.

Remark 11.2 (Similar matrices)**.** Two matrices A and $A_B = B^{-1}AB$ represent the *same* linear map in different bases. Such matrices are called *similar*. Similarity is an equivalence relation; finding the simplest representative in each equivalence class is the central problem of §4.

Python: change of basis

```
import numpy as np

A = np.array([[2., 1.],
              [0., 3.]]) # matrix of L in standard basis

B = np.array([[1., 1.],
              [0., 1.]]) # new basis (columns are basis vectors)

A_B = np.linalg.inv(B) @ A @ B # matrix of L in basis B
print("A in standard basis:\n", A)
print("A in new basis B:\n", A_B)

# Verify: same eigenvalues (similarity preserves eigenvalues)
print("eigenvalues of A: ", np.linalg.eigvals(A))
print("eigenvalues of A_B:", np.linalg.eigvals(A_B)) # same
```

Derivation of the change-of-basis formula

Let $P = [b_1' \mid b_2' \mid \cdots \mid b_n']$ be the change-of-basis matrix (columns = new basis vectors, expressed in the standard basis). A vector with standard coordinates x has B-coordinates $x' = P^{-1}x$; conversely $x = Px'$.

Remark 11.3 (P is always invertible)**.** The columns of P form a basis, hence are linearly independent, hence P is square and invertible. The change-of-basis matrix always has an inverse.

The commutative diagram:

$$\begin{array}{ccc} x \in \mathbb{R}^n_{\text{std}} & \xrightarrow{A} & y \in \mathbb{R}^n_{\text{std}} \\ {\scriptstyle P}\uparrow & & \uparrow{\scriptstyle P} \\ x' \in \mathbb{R}^n_B & \xrightarrow{A'} & y' \in \mathbb{R}^n_B \end{array}$$

From $y = Ax$ and $x = Px'$, $y = Py'$:

$$Py' = A\,Px' \implies y' = P^{-1}AP\,x' \implies \boxed{A' = P^{-1}AP.}$$

Equivalently: $A = PA'P^{-1}$, so the standard-basis matrix is recovered from the B-basis matrix by conjugation with P.

Remark 11.4 (Two equivalent forms)**.**

- $A' = P^{-1}AP$ — matrix of L in basis B, given standard-basis matrix A.
- $A = PA'P^{-1}$ — standard-basis matrix, given B-basis matrix A'.

Students often confuse which way the P goes; checking with a simple example (as below) is the safest remedy.

Python: verifying $A' = P^{-1}AP$

```python
import numpy as np

A = np.array([[3., 1.],
              [0., 2.]]) # matrix of L in standard basis

P = np.array([[1., 2.],
              [1., 3.]]) # columns = new basis vectors (invertible)

A_prime = np.linalg.inv(P) @ A @ P # matrix of L in basis B

# Same linear map: A applied to Px' equals P times A'x'
x_prime = np.array([1., -1.])
print("A(Px') :", A @ (P @ x_prime))
print("P(A'x'):", P @ (A_prime @ x_prime)) # must match

# Similarity preserves eigenvalues
print("eig(A) :", np.linalg.eigvals(A))
print("eig(A_prime):", np.linalg.eigvals(A_prime)) # same
```

12 Eigenvalues and Diagonalization (§4.2)

Central question of §4. Given $A \in \mathbb{R}^{n\times n}$, is there an invertible $P \in \mathbb{R}^{n\times n}$ such that $P^{-1}AP$ has a simpler structure — e.g. diagonal, upper triangular?

§4.2 Eigenvectors and eigenvalues

Definition 12.1. Let $L : V \to V$ be a linear transform on a vector space V over F. A vector $v \neq 0$, $v \in V$ is an *eigenvector* of L with *eigenvalue* $\lambda \in F$ iff

$$L(v) = \lambda v.$$

The condition $v \neq 0$ is essential: $L(0) = \lambda \cdot 0$ holds for every λ and would make eigenvalues meaningless.

Example 12.1. $A = \begin{pmatrix} 1 & 2 \\ 0 & 3 \end{pmatrix}$ has eigenvectors $v_1 = (1,0)^T$ and $v_2 = (1,1)^T$:

$$Av_1 = (1,0)^T = 1 \cdot v_1, \qquad Av_2 = (3,3)^T = 3 \cdot (1,1)^T = 3v_2.$$

So $\lambda_1 = 1$ and $\lambda_2 = 3$.

Proposition 12.1 (Eigenvalue $\Leftrightarrow$ non-invertible shift)**.** *λ is an eigenvalue of L if and only if $\ker(L - \lambda I) \neq \{0\}$, i.e. $L - \lambda I$ is **not** invertible.*

Proof. $L(v) = \lambda v \Leftrightarrow (L - \lambda I)(v) = 0 \Leftrightarrow v \in \ker(L - \lambda I)$. Since $v \neq 0$, the kernel is non-trivial. By the Im-Ker Theorem, $L - \lambda I$ is then not injective, hence not invertible (for finite-dimensional V). □

Remark 12.1 (Forward: eigenvectors give diagonalisation)**.** If L has n linearly independent eigenvectors $v_1, \ldots, v_n$ with eigenvalues $\lambda_1, \ldots, \lambda_n$, then setting $P = [v_1 | \cdots | v_n]$:

$$P^{-1}AP = \operatorname{diag}(\lambda_1, \ldots, \lambda_n).$$

Eigenvectors are exactly the basis that answers the simplification question. When such a basis exists, L is called *diagonalisable*.

Python: eigenvalues and eigenvectors

```
import numpy as np

A = np.array([[1., 2.],
              [0., 3.]])
vals, vecs = np.linalg.eig(A)
print("eigenvalues:", vals) # [1. 3.]
print("eigenvectors:\n", vecs) # columns (normalised by numpy)

# Verify Av = lambda v for each eigenvector
for i in range(len(vals)):
    v = vecs[:, i]
    print(f"Av{i+1} = lam*v{i+1}:", np.allclose(A @ v, vals[i] * v)) # True

# Diagonalisation: P^{-1}AP = diag(lambda_1, lambda_2)
P = vecs
D = np.linalg.inv(P) @ A @ P
print("P^{-1}AP:\n", D.round(10)) # diagonal with [1, 3]
```

Building toward diagonalisation

Since $v \neq 0$, we can complete it to a basis $B = \{v, u_2, \ldots, u_n\}$. The matrix of L in basis B is formed by columns $[L]_B = [L(u_1) \mid L(u_2) \mid \cdots \mid L(u_n)]$. Since $L(v) = \lambda v$ (i.e. $L(u_1) = \lambda_1 u_1$ in B-coordinates), the first column is $\lambda_1 e_1$:

$$[L]_B = \begin{pmatrix} \lambda_1 & * & \cdots & * \\ 0 & & & \\ \vdots & & * & \\ 0 & & & \end{pmatrix}.$$

If u_2 is also an eigenvector with $L(u_2) = \lambda_2 u_2$, $\lambda_2 \neq \lambda_1$, the matrix simplifies further:

$$[L]_B = \begin{pmatrix} \lambda_1 & 0 & * & \cdots \\ 0 & \lambda_2 & * & \cdots \\ 0 & 0 & & \\ \vdots & & & * \end{pmatrix}.$$

Continuing: if all n basis vectors are eigenvectors, $[L]_B$ is diagonal.

Remark 12.2 (Invariant subspaces)**.** The zero pattern (zeros below each diagonal eigenvalue) reflects that L maps $\operatorname{span}\{u_1, \ldots, u_k\}$ into itself for each k — these are *invariant subspaces* of L. This concept will reappear in the Jordan normal form and SVD.

Proposition 12.2 (4.2.1)**.** *If $L(u_1) = \lambda_1 u_1$ and $L(u_2) = \lambda_2 u_2$ with $\lambda_1 \neq \lambda_2$, then u_1 and u_2 are linearly independent.*

Proof. Suppose $c_1 u_1 + c_2 u_2 = 0$. Apply L: $\lambda_1 c_1 u_1 + \lambda_2 c_2 u_2 = 0$. Subtract λ_1 times the first equation: $(\lambda_2 - \lambda_1) c_2 u_2 = 0$. Since $u_2 \neq 0$ and $\lambda_1 \neq \lambda_2$, we get $c_2 = 0$, then $c_1 = 0$. □

Corollary 12.1. *Eigenvectors with pairwise distinct eigenvalues are linearly independent. Hence any $A \in \mathbb{R}^{n \times n}$ with n distinct eigenvalues is automatically diagonalisable.*

The corollary says: as long as we keep finding eigenvectors with distinct eigenvalues, we accumulate linearly independent vectors. What is the structure of $[L]_B$ when we have k such eigenvectors but cannot (yet) fill out the remaining $r = n - k$ slots with eigenvectors?

Proposition 12.3 (Partial diagonalisation)**.** *Let* $\dim V = n$*. Suppose* $u_1, \dots, u_k$ *are eigenvectors of* L *with pairwise distinct eigenvalues* $\lambda_1, \dots, \lambda_k$*. Extend to a basis*

$$B = \{u_1, \dots, u_k,\ v_1, \dots, v_r\}, \quad k + r = n.$$

Then, in the basis B*,*

$$[L]_B = \begin{pmatrix} \lambda_1 & & & * & \cdots & * \\ & \ddots & & \vdots & & \vdots \\ & & \lambda_k & * & \cdots & * \\ 0 & \cdots & 0 & & & \\ \vdots & & \vdots & & * & \\ 0 & \cdots & 0 & & & \end{pmatrix} = \begin{pmatrix} \Lambda & * \\ 0 & * \end{pmatrix},$$

where $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_k)$*, the lower-left* $r \times k$ *block is zero, and the right-hand* $*$ *blocks are unconstrained.*

Proof. For each $i \le k$: in B-coordinates, $L(u_i) = \lambda_i u_i$ means the i-th column of $[L]_B$ is $\lambda_i e_i$ (eigenvalue in position i, zeros elsewhere). In particular all entries below row k in those columns are zero. The columns corresponding to $v_1, \dots, v_r$ are unconstrained (we imposed no condition on the v_j). □

Remark 12.3 (Three cases by the value of k)**.**

- $k = n$: every basis vector is an eigenvector, $[L]_B = \Lambda$ is diagonal — L is *diagonalisable*.
- $k < n$: the lower-right $r \times r$ block $*$ captures the "non-diagonalisable part". Understanding this block leads to the *Jordan normal form*.
- $k = 0$: no eigenvectors at all over $\mathbb{R}$ (e.g. a rotation in $\mathbb{R}^2$) — must work over $\mathbb{C}$ to find them.

Python: partial diagonalisation and block structure

```
import numpy as np

# Matrix with only k=2 linearly independent eigenvectors out of n=3
# (repeated eigenvalue lambda=2 with 1-dim eigenspace)
A = np.array([[2., 1., 0.],
              [0., 2., 0.],
              [0., 0., 3.]], float) # Jordan block in upper-left

vals, vecs = np.linalg.eig(A)
print("eigenvalues:", vals) # [2. 2. 3.]
print("rank of eigenvector matrix:", np.linalg.matrix_rank(vecs)) # 2 < 3

# Distinct-eigenvalue case: partial diagonalisation is exact diagonalisation
B = np.array([[2., 1., 3.],
              [0., 4., 2.],
```

```
            [0., 0., 6.]], float)
vals2, vecs2 = np.linalg.eig(B)
D = np.linalg.inv(vecs2) @ B @ vecs2
print("P^{-1}AP (should be diagonal):\n", D.round(8))

# Verify zero pattern: lower-left block is 0
print("lower-left entries near zero:", np.allclose(D[1:, 0], 0)) # True
```

Open questions

Three questions drive the rest of the course:

1. **Which other matrices are diagonalizable?** Distinct eigenvalues are *sufficient* but not necessary. The obstruction lives in the $r \times r$ block of the partial diagonalisation (Prop. 12.3): a matrix is diagonalizable iff that block can itself be diagonalized, i.e. iff V has a basis of eigenvectors. Key positive answer: *symmetric* (or, in general, *normal*) matrices are always diagonalizable — the spectral theorem, coming next.

2. **How do we find eigenvalues and eigenvectors?** Theoretically: solve $\det(A - \lambda I) = 0$ for λ, then find $\ker(A - \lambda I)$ for each λ. In practice: for $n \geq 5$ this is numerically unstable and wasteful; eigenvalues are computed via iterative algorithms (QR iteration), not by expanding a determinant.

3. **Can we guarantee their existence?** Over $\mathbb{C}$: yes — the Fundamental Theorem of Algebra says $\det(A - \lambda I) = 0$ always has n roots (counted with multiplicity), so every matrix has at least one complex eigenvalue. Over $\mathbb{R}$: not in general — a rotation by 90 in $\mathbb{R}^2$ has no real eigenvalues. The guarantee is restored for symmetric matrices: all eigenvalues are real (proved via PSD, next section).

Transition. Symmetric matrices $A = A^T$ form the class where all three questions have clean answers: they are always diagonalizable, their eigenvalues are always real, and there is a basis of orthonormal eigenvectors. This is the *spectral theorem*, and it follows from the theory of positive semidefinite matrices.

Putting it all together: if $L\colon V \to V$ has $n = \dim V$ distinct eigenvalues $\lambda_1, \ldots, \lambda_n$ with eigenvectors $u_1, \ldots, u_n$, then $B = \{u_1, \ldots, u_n\}$ is a basis (by Corollary above) and

$$[L]_B = \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n).$$

The structure of L in the eigenbasis B is maximally simple, compared with the standard-basis matrix $[L]_I = A$, which is generically dense.

Definition 12.2 (Diagonalizable matrix)**.** A matrix $A \in \mathbb{R}^{n \times n}$ is *diagonalizable* if there exists an invertible $P \in \mathbb{R}^{n \times n}$ such that

$$P^{-1}AP = \operatorname{diag}(\mu_1, \ldots, \mu_n).$$

Note: the μ_i need *not* be distinct. (Example: $A = I$ is diagonalized by any invertible P, with all $\mu_i = 1$.)

Remark 12.4 (Convention: $P^{-1}AP$ vs PAP^{-1})**.** Both conventions appear in the literature. Our §4.1 formula $A' = P^{-1}AP$ (columns of P are the new basis vectors expressed in the old basis) is the one we use throughout. If P is the matrix whose i-th column is eigenvector u_i, then $P^{-1}AP = \Lambda$ — consistent with the change-of-basis formula.

Proposition 12.4 (4.2.2)**.** *If $A \in \mathbb{R}^{n\times n}$ has n distinct eigenvalues $\lambda_1, \dots, \lambda_n$ ($\lambda_i \neq \lambda_j$ for $i \neq j$), then A is diagonalizable:*

$$P^{-1}AP = \operatorname{diag}(\lambda_1, \dots, \lambda_n),$$

where $P = [u_1 \mid \cdots \mid u_n]$ is the matrix of eigenvectors.

Proof. By Corollary to Prop. 4.2.1, the n eigenvectors are linearly independent, hence form a basis B. In that basis $[L]_B$ is diagonal (Prop. 12.3 with $k = n$, $r = 0$). The change-of-basis formula gives $[L]_B = P^{-1}AP = \Lambda$. □

Definition 12.3 (Similar matrices)**.** Two matrices $A, A' \in \mathbb{R}^{n\times n}$ are *similar* (written $A' \sim A$) if $A' = P^{-1}AP$ for some invertible P. Similarity is an equivalence relation (reflexive, symmetric, transitive). Diagonalizable matrices are precisely those similar to a diagonal matrix.

Remark 12.5 (Finding eigenvalues in practice)**.** The theoretical criterion $\ker(A - \lambda I) \neq \{0\}$ can be phrased as $\det(A - \lambda I) = 0$. For small matrices ($n \leq 4$) this gives a polynomial whose roots are the eigenvalues — a clean formula. For $n \geq 5$ the Abel–Ruffini theorem says no closed-form root formula exists; for large n computing the characteristic polynomial is numerically unstable and wasteful. In practice, eigenvalues are found via iterative algorithms (QR iteration), not by expanding a determinant.

Python: diagonalization

```python
import numpy as np

# Diagonalization via eigenvectors
A = np.array([[1., 2.],
              [0., 3.]])
vals, P = np.linalg.eig(A)
Lambda = np.linalg.inv(P) @ A @ P
print("P^{-1}AP =\n", Lambda.round(10)) # diagonal: [1, 3]

# Similarity is an equivalence relation:
# A ~ A (reflexive: P = I)
print("A ~ A:", np.allclose(np.linalg.inv(np.eye(2)) @ A @ np.eye(2), A))

# Eigenvalues are preserved under similarity
A2 = P @ np.diag([5., 7.]) @ np.linalg.inv(P) # construct a similar matrix
print("eigenvalues of similar matrix:", np.linalg.eigvals(A2).round(8)) # [5, 7]
```

13 Positive Semidefinite Matrices (§4.3)

Definition and first properties

Definition 13.1 (PSD via factorization)**.** A matrix $A \in \mathbb{R}^{n\times n}$ is called *positive semidefinite* (PSD) if

$$A = B^T B \quad \text{for some } B \in \mathbb{R}^{m\times n}.$$

It is *positive definite* (PD) if additionally $\ker B = \{0\}$.

Remark 13.1 (Two common definitions — not always equivalent)**.** Several definitions of PSD appear in the literature:

1. **(Our definition)** $A = B^T B$ for some B.
2. **(Quadratic form)** $\forall x \in \mathbb{R}^n,\ x^T A x \geq 0$. This does *not* require A to be symmetric.
3. **(Symmetric + quadratic form)** $A = A^T$ and $x^T A x \geq 0$ for all x.

Our definition (1) automatically gives $A^T = (B^T B)^T = B^T B = A$, so it implies symmetry. Definitions (1) and (3) are equivalent for symmetric matrices; definition (2) is strictly weaker. Throughout these notes we use definition (1)/(3): PSD means symmetric with $x^T A x \geq 0$.

Quadratic forms. The expression $f(x) = x^T A x$ is called a *quadratic form*. For a PSD matrix $A = B^T B$:

$$f(x) = x^T B^T B x = (Bx)^T (Bx) = \|Bx\|^2 \geq 0,$$

using the ℓ^2-norm $\|y\|^2 = y_1^2 + \cdots + y_m^2$. So $f(x) \geq 0$ with minimum $f(x) = 0$ achieved precisely on $\ker B$.

Key example. $A^T A$ is always PSD: take $B = A$, then $A^T A = B^T B$. This is the matrix in the normal equations.

Kernel of a PSD matrix

Proposition 13.1. *If $A = B^T B$, then $\ker B = \ker A$.*

Proof. This is the same argument as in §3 (normal equations): $Ax = 0$ iff $x^T A x = 0$ iff $\|Bx\|^2 = 0$ iff $Bx = 0$. More explicitly: $\ker B \subseteq \ker A$ is clear ($Bx = 0 \Rightarrow B^T B x = 0$). Conversely, $Ax = B^T B x = 0$ implies $x^T B^T B x = \|Bx\|^2 = 0$, so $Bx = 0$. □

Consequence. Let $V = \mathbb{R}^n = \ker A \oplus (\ker A)^\perp$. On $(\ker A)^\perp$, we have $f(x) = \|Bx\|^2 = 0 \Rightarrow x = 0$, so A is *positive definite on* $(\ker A)^\perp$.

Python: three equivalent characterizations

```python
import numpy as np

rng = np.random.default_rng(42)
B = rng.standard_normal((4, 3)) # tall matrix
A = B.T @ B # 3x3 PSD by construction

# (1) Check symmetry
print("symmetric:", np.allclose(A, A.T))

# (2) Check all eigenvalues >= 0
vals = np.linalg.eigvalsh(A) # eigvalsh: symmetric matrix
print("eigenvalues:", vals.round(8)) # all >= 0

# (3) Check x^T A x >= 0 for random x
xs = rng.standard_normal((1000, 3))
quad_forms = np.einsum('ni,ij,nj->n', xs, A, xs)
print("all x^TAx >= 0:", np.all(quad_forms >= -1e-12))

# (4) ker B = ker A
from scipy.linalg import null_space
print("null_space(B) approx empty (full rank B):", null_space(B).shape[1])
```

Invariant subspaces and restrictions

Definition 13.2 (Invariant subspace)**.** A subspace $U \leq V$ is *invariant under* $L \in \mathcal{L}(V, V)$ (or *L-invariant*) if $L(U) \subseteq U$, i.e. $L(u) \in U$ for every $u \in U$.

Example 13.1. $\ker A$ is always A-invariant: if $Au = 0$ then $A(Au) = A \cdot 0 = 0$, so $Au \in \ker A$.

Definition 13.3 (Restriction)**.** If $U \leq V$ is L-invariant, the *restriction* $L\big|_U \in \mathcal{L}(U, U)$ is defined by

$$L\big|_U(u) = L(u) \quad \forall\, u \in U.$$

The restriction is well-defined precisely because U is invariant.

Symmetric matrices preserve orthogonal complements

Lemma 13.1. *Let $A \in \mathbb{R}^{n\times n}$ be symmetric ($A^T = A$) and let $U \leq \mathbb{R}^n$ be A-invariant. Then $U^\perp$ is also A-invariant.*

Proof. Take $w \in U^\perp$, so $\langle w, u\rangle = 0$ for all $u \in U$. We must show $Aw \in U^\perp$, i.e. $\langle Aw, u\rangle = 0$ for all $u \in U$:

$$\langle Aw, u\rangle = (Aw)^T u = w^T A^T u = w^T Au.$$

Since U is A-invariant, $Au \in U$; since $w \in U^\perp$, $w^T(Au) = 0$. Hence $\langle Aw, u\rangle = 0$, so $Aw \in U^\perp$. $\square$

Remark 13.2 (Inductive sketch of the spectral theorem)**.** Lemma 13.1 is the engine of the spectral theorem. Let A be symmetric and take any eigenvalue λ with eigenvector v. Set $U = \operatorname{span}\{v\}$; this is A-invariant. By the lemma, $U^\perp$ is also A-invariant, so $A\big|_{U^\perp}$ is a well-defined symmetric map on an $(n-1)$-dimensional space. Apply the argument inductively: at each step peel off one eigenvalue and restrict to the orthogonal complement. The result is an orthonormal basis of eigenvectors — the spectral theorem.

Python: $U^\perp$ invariance for a symmetric matrix

```python
import numpy as np

# Build a symmetric matrix
rng = np.random.default_rng(0)
M = rng.standard_normal((4, 4))
A = M + M.T # symmetric

# Find an eigenspace (A-invariant subspace U)
vals, vecs = np.linalg.eigh(A) # orthonormal eigenvectors
U = vecs[:, :2] # span of first two eigenvectors

# Orthogonal complement U_perp: span of remaining eigenvectors
U_perp = vecs[:, 2:]

# Check A maps U_perp into U_perp: project A @ w onto U and check it's ~0
for i in range(U_perp.shape[1]):
    w = U_perp[:, i]
    Aw = A @ w
    component_in_U = U @ (U.T @ Aw) # projection of Aw onto U
    print(f"||proj_U(A w_{i})||: {np.linalg.norm(component_in_U):.2e}")
    # should be ~0 (machine precision)
```

The spectral theorem: proof via constrained optimisation

Goal. We want to show that every symmetric $A \in \mathbb{R}^{n\times n}$ has a real eigenvector. Combined with the inductive argument (Lemma 13.1), this gives an orthonormal eigenbasis — the spectral theorem.

On $U^\perp = (\ker A)^\perp$ we have $f(x) = x^T Ax > 0$ for all $x \neq 0$, so the quadratic form is a "bowl" with global minimum $f(0) = 0$. (Note: this holds on $U^\perp$ specifically, not on all of $\mathbb{R}^n$.) Consider the constrained maximum:

$$\max_{\|x\|=1} f(x).$$

The unit sphere $\{x : \|x\| = 1\}$ is compact and f is continuous, so the maximum is attained at some v with $\|v\| = 1$.

Proposition 13.2 (Maximizer is an eigenvector)**.** *Let $v \in \mathbb{R}^n$, $\|v\| = 1$, satisfy $f(v) = \max_{\|x\|=1} f(x)$. Then $Av = \lambda v$ with $\lambda = f(v) \geq 0$.*

Proof. For any $w \in \langle v\rangle^\perp$, consider the unit-normalised path

$$u(t) = \frac{v + tw}{\|v + tw\|}, \quad g(t) = f(u(t)) = \frac{(v + tw)^T A(v + tw)}{\|v + tw\|^2}.$$

Since $w \perp v$ we have $\|v + tw\|^2 = \|v\|^2 + t^2\|w\|^2 = 1 + t^2\|w\|^2$, so

$$g(t) = \frac{v^T Av + 2t\, w^T Av + t^2\, w^T Aw}{1 + t^2\|w\|^2}.$$

Since v is a maximizer on the unit sphere, $g'(0) = 0$.

Exercise. Show using the quotient rule (and $w^T v = 0$, $\|v\| = 1$) that $g'(0) = 2w^T Av$.

Hence $w^T Av = 0$ for all $w \in \langle v\rangle^\perp$, i.e. $Av \perp w$ for every $w \perp v$. This means $Av \in \langle v\rangle^{\perp\perp} = \langle v\rangle$, so $Av = \lambda v$ for some $\lambda \in \mathbb{R}$. Taking $w = v$: $\lambda = v^T Av = f(v) \geq 0$ (since A is PSD). □

Corollary 13.1 (PSD $\Rightarrow$ eigenvalues ≥ 0)**.** *If A is PSD then every eigenvalue satisfies $\lambda \geq 0$.*

Proof. If $Av = \lambda v$, $v \neq 0$, then $\lambda\|v\|^2 = v^T Av \geq 0$, so $\lambda \geq 0$. □

Theorem 13.1 (Spectral theorem)**.** *Let $A \in \mathbb{R}^{n\times n}$ be symmetric ($A = A^T$). Then:*

1. *All eigenvalues of A are real.*
2. *A has an orthonormal basis of eigenvectors: $A = Q\Lambda Q^T$ where Q is orthogonal and $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$.*
3. *If additionally A is PSD, then $\lambda_i \geq 0$ for all i.*

Proof (induction via Lemma 13.1). The proposition gives a unit eigenvector v_1 with $\lambda_1 = f(v_1) \geq 0$ (the largest eigenvalue). Set $W = \langle v_1\rangle$; by Lemma 13.1, $W^\perp$ is A-invariant and inherits symmetry. Apply the proposition to $A\big|_{W^\perp}$, obtaining v_2 with $v_2 \perp v_1$. Repeat n times to produce an orthonormal eigenbasis $\{v_1, \ldots, v_n\}$. Setting $Q = [v_1 \mid \cdots \mid v_n]$ gives $AQ = Q\Lambda$, i.e. $A = Q\Lambda Q^T$. □

Definition 13.4 (Eigenspace)**.** For $L \in \mathcal{L}(V, V)$ and $\lambda \in \mathbb{F}$, the λ-*eigenspace* of L is

$$E_\lambda = \{v \in V : L(v) = \lambda v\} = \ker(L - \lambda\,\mathrm{id}).$$

Note $E_\lambda \leq V$. The dimension $\dim E_\lambda$ is the *geometric multiplicity* of λ.

Theorem 13.2 (4.3.1 — Spectral theorem for PSD matrices)**.** *Let $A \in \mathbb{R}^{n\times n}$ be PSD. Then there exists an orthonormal eigenbasis $\{v_1, \dots, v_n\}$ (i.e. $v_i^T v_j = \delta_{ij}$) such that $Av_i = \lambda_i v_i$, $\lambda_i \geq 0$. Setting $P = [v_1 \mid \cdots \mid v_n]$ (orthogonal, $P^T P = I$, $P^{-1} = P^T$):*

$$P^T AP = \operatorname{diag}(\lambda_1, \dots, \lambda_n), \quad \lambda_i \geq 0.$$

Remark 13.3 ($P^T AP$ vs $P^{-1}AP$)**.** Because P is orthogonal ($P^T = P^{-1}$), the change-of-basis formula $P^{-1}AP = \Lambda$ simplifies to $P^T AP = \Lambda$. This is a strictly stronger form of diagonalizability: the eigenbasis is not only a basis but an *orthonormal* one. General diagonalizable matrices only have $P^{-1}AP = \Lambda$ with P possibly non-orthogonal.

Remark 13.4 (Forward pointer: SVD)**.** For a PSD matrix, the SVD coincides with the eigendecomposition: $A = Q\Lambda Q^T$ with $\Lambda = \Sigma^2$. For a general rectangular matrix $A \in \mathbb{R}^{m\times n}$, the SVD $A = U\Sigma V^T$ replaces the eigendecomposition — applying the spectral theorem to $A^T A$ and AA^T simultaneously. This is the subject of §4.4.

Python: largest eigenvector as constrained maximum

```python
import numpy as np

rng = np.random.default_rng(7)
M = rng.standard_normal((4, 4))
A = M + M.T # symmetric 4x4

# True largest eigenvalue and eigenvector
vals, vecs = np.linalg.eigh(A) # eigenvalues in ascending order
lambda_max = vals[-1]
v_max = vecs[:, -1]
print(f"lambda_max = {lambda_max:.6f}, f(v_max) = {v_max @ A @ v_max:.6f}")

# Sample random unit vectors and compute f(x) = x^T A x
N = 100_000
X = rng.standard_normal((N, 4))
X /= np.linalg.norm(X, axis=1, keepdims=True) # project onto unit sphere
fvals = np.einsum('ni,ij,nj->n', X, A, X)

print(f"max f(x) over {N} samples: {fvals.max():.6f}")
print(f"ratio (should be close to 1): {fvals.max() / lambda_max:.6f}")

# ------------------------------------------------------------------
# Theorem 4.3.1: P^T A P = Lambda, P^T P = I (PSD matrix)
B = rng.standard_normal((5, 4))
A_psd = B.T @ B # 4x4 PSD by construction
vals_psd, Q = np.linalg.eigh(A_psd) # Q orthogonal, vals >= 0

print("\nSpectral theorem verification:")
print("eigenvalues (>= 0):", vals_psd.round(8))
print("P^T P = I:", np.allclose(Q.T @ Q, np.eye(4)))
Lambda = Q.T @ A_psd @ Q
print("P^T A P diagonal:", np.allclose(Lambda, np.diag(vals_psd)))
print("diagonal entries:", np.diag(Lambda).round(8))
```

14 Trace

Definition 14.1 (Axiomatic characterization)**.** The *trace* is the unique linear map $\mathrm{tr} : \mathbb{F}^{n\times n} \to \mathbb{F}$ satisfying:

1. $\mathrm{tr}(AB) = \mathrm{tr}(BA)$ for all A, B.
2. $\mathrm{tr}(I) = n$.

Proposition 14.1. $\mathrm{tr}(A) = \sum_i a_{ii}$ *(sum of diagonal entries).*

Proof. Let E_{ij} be the standard matrix units. By linearity, it suffices to compute $\mathrm{tr}(E_{ij})$. If $i \neq j$: $\mathrm{tr}(E_{ij}) = \mathrm{tr}(E_{ii}E_{ij}) = \mathrm{tr}(E_{ij}E_{ii}) = 0$. For $i = j$: for any i, j there is a permutation matrix P with $P^{-1}E_{jj}P = E_{ii}$. By cyclicity, $\mathrm{tr}(E_{ii}) = \mathrm{tr}(P^{-1}E_{jj}P) = \mathrm{tr}(E_{jj})$. So all $\mathrm{tr}(E_{ii})$ are equal; since $I = \sum_i E_{ii}$ and $\mathrm{tr}(I) = n$, we get $\mathrm{tr}(E_{ii}) = 1$. Hence $\mathrm{tr}(A) = \sum_i a_{ii}$. □

Proposition 14.2 (Similarity invariance)**.** $\mathrm{tr}(P^{-1}AP) = \mathrm{tr}(A)$.

Proof. $\mathrm{tr}(P^{-1}AP) = \mathrm{tr}(APP^{-1}) = \mathrm{tr}(A)$. □

Corollary 14.1. *If A is diagonalizable with eigenvalues $\lambda_1, \ldots, \lambda_n$, then* $\mathrm{tr}(A) = \sum_i \lambda_i$.

15 Singular Value Decomposition (§4.4)

Motivation: symmetric is not enough

The spectral theorem (Thm. 13.1) gives a complete orthonormal eigenbasis for *symmetric* matrices. Most matrices in practice — data matrices, regression matrices, image matrices — are rectangular and not symmetric. The SVD extends the spectral theorem to every $A \in \mathbb{R}^{m\times n}$.

Step 1: A^TA is PSD

Lemma 15.1. *For any $A \in \mathbb{R}^{m\times n}$, the matrices $A^TA \in \mathbb{R}^{n\times n}$ and $AA^T \in \mathbb{R}^{m\times m}$ are both symmetric PSD.*

Proof. $(A^TA)^T = A^T(A^T)^T = A^TA$, so symmetric. $x^T(A^TA)x = (Ax)^T(Ax) = \|Ax\|^2 \geq 0$, so PSD. Identical argument for AA^T. □

By the spectral theorem, A^TA has an orthonormal eigenbasis $\{v_1, \ldots, v_n\}$ with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$.

Step 2: right singular vectors and singular values

Write $A^TAv_i = \lambda_i v_i$. Define the *singular values*

$$\sigma_i = \sqrt{\lambda_i} \geq 0, \qquad i = 1, \ldots, n.$$

Let $r = \mathrm{rank}(A)$. Then $\sigma_1 \geq \cdots \geq \sigma_r > 0 = \sigma_{r+1} = \cdots = \sigma_n$ (since $\ker(A^TA) = \ker A$ by §4.3, so $\lambda_i = 0 \Leftrightarrow v_i \in \ker A$).

The v_i are the *right singular vectors* of A.

Step 3: left singular vectors

Lemma 15.2. *For $i \le r$, define $u_i = Av_i/\sigma_i$. Then:*

1. *u_i is a unit vector: $\|u_i\| = 1$.*
2. *$u_i \perp u_j$ for $i \neq j$.*
3. *$AA^T u_i = \sigma_i^2 u_i$ (so u_i is an eigenvector of AA^T).*

Proof. (1): $\|u_i\|^2 = \|Av_i\|^2/\sigma_i^2 = v_i^T A^T A v_i/\sigma_i^2 = \sigma_i^2/\sigma_i^2 = 1$.
(2): $\langle u_i, u_j\rangle = (Av_i)^T(Av_j)/(\sigma_i\sigma_j) = v_i^T A^T A v_j/(\sigma_i\sigma_j) = \sigma_j^2 v_i^T v_j/(\sigma_i\sigma_j) = 0$ (since $v_i \perp v_j$).
(3): $AA^T u_i = AA^T(Av_i/\sigma_i) = A(A^T A v_i)/\sigma_i = A(\sigma_i^2 v_i)/\sigma_i = \sigma_i\, Av_i = \sigma_i^2 u_i$. □

Extend $\{u_1, \dots, u_r\}$ to a full orthonormal basis $\{u_1, \dots, u_m\}$ of $\mathbb{R}^m$ (the remaining $u_{r+1}, \dots, u_m$ span $\ker(A^T)$). These are the *left singular vectors*.

Step 4: assembling the SVD

Set

$$U = [u_1 \mid \cdots \mid u_m] \in \mathbb{R}^{m\times m}, \quad V = [v_1 \mid \cdots \mid v_n] \in \mathbb{R}^{n\times n}, \quad \Sigma \in \mathbb{R}^{m\times n} \text{ (“diagonal”: } \Sigma_{ii} = \sigma_i).$$

Theorem 15.1 (Singular Value Decomposition)**.** *For any $A \in \mathbb{R}^{m\times n}$ with $\operatorname{rank}(A) = r$,*

$$A = U\Sigma V^T,$$

where $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ are orthogonal, and $\Sigma \in \mathbb{R}^{m\times n}$ has $\sigma_1 \ge \cdots \ge \sigma_r > 0$ on its main diagonal and zeros elsewhere.

Proof. Check column by column. For $i \le r$: $(U\Sigma V^T)v_i = U(\Sigma e_i) = \sigma_i\, Ue_i = \sigma_i u_i = Av_i$. For $i > r$: $Av_i = 0$ (since $v_i \in \ker A$), and $(U\Sigma V^T)v_i = \sigma_i u_i = 0$. Since $\{v_i\}$ is a basis of $\mathbb{R}^n$, $A = U\Sigma V^T$. □

Remark 15.1 (Four fundamental subspaces via SVD)**.** The SVD makes the four subspaces explicit:

$$\begin{aligned}
\operatorname{col}(A) &= \operatorname{span}\{u_1, \dots, u_r\},\\
\ker(A^T) &= \operatorname{span}\{u_{r+1}, \dots, u_m\},\\
\operatorname{row}(A) &= \operatorname{span}\{v_1, \dots, v_r\},\\
\ker(A) &= \operatorname{span}\{v_{r+1}, \dots, v_n\}.
\end{aligned}$$

Remark 15.2 (Geometric picture)**.** A factors as three simple operations:

$$\mathbb{R}^n \xrightarrow{V^T} \text{rotate} \xrightarrow{\Sigma} \text{stretch (axis } i \text{ by } \sigma_i) \xrightarrow{U} \text{rotate} \in \mathbb{R}^m.$$

The image of the unit sphere under A is an ellipsoid with semi-axis lengths $\sigma_1 \ge \cdots \ge \sigma_r$ along directions $u_1, \dots, u_r$.

Low-rank approximation and the Eckart–Young theorem

Outer-product expansion

Writing $U\Sigma V^T$ column by column gives the *outer-product form*:

$$A = \sum_{i=1}^{r} \sigma_i \, u_i v_i^T.$$

Each term $\sigma_i u_i v_i^T$ is a rank-1 matrix. The *rank-k truncation* keeps only the k largest singular values:

$$A_k = \sum_{i=1}^{k} \sigma_i \, u_i v_i^T, \qquad k \le r.$$

The Eckart–Young theorem

Theorem 15.2 (Eckart–Young)**.**

1. (Error formula) $\|A - A_k\|_F^2 = \sigma_{k+1}^2 + \cdots + \sigma_r^2$.
2. (Optimality) *A_k is the closest rank-$\le k$ matrix to A in the Frobenius norm:* $A_k = \operatorname{argmin}_{\operatorname{rank}(B)\le k} \|A - B\|_F$.

Proof. **Part 1.** $A - A_k = \sum_{i=k+1}^{r} \sigma_i u_i v_i^T$. The rank-1 terms are Frobenius-orthogonal: $\langle u_i v_i^T, u_j v_j^T \rangle_F = \operatorname{tr}(v_i u_i^T u_j v_j^T) = (u_i^T u_j)(v_i^T v_j) = \delta_{ij}$. Hence $\|A - A_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2$.

Part 2. Let B have rank $\le k$; we show $\|A - B\|_F^2 \ge \sum_{i=k+1}^{r} \sigma_i^2$.

Step 1: find $r-k$ test vectors. Since $\dim\ker(B) \ge n - k$ and $\dim\operatorname{span}\{v_1, \ldots, v_r\} = r$,

$$\dim(\ker B \cap \operatorname{span}\{v_1, \ldots, v_r\}) \ge (n-k) + r - n = r - k.$$

Pick an orthonormal set $\{w_{k+1}, \ldots, w_r\} \subset \ker(B) \cap \operatorname{span}\{v_1, \ldots, v_r\}$.

Step 2: lower-bound the error. Since $Bw_j = 0$:

$$\|A - B\|_F^2 \;\ge\; \sum_{j=k+1}^{r} \|(A-B)w_j\|^2 = \sum_{j=k+1}^{r} \|Aw_j\|^2.$$

Step 3: expand in the v_i basis. Write $w_j = \sum_{i=1}^{r} \alpha_{ji} v_i$. The $(r-k) \times r$ matrix $\Phi = [\alpha_{ji}]$ has orthonormal rows ($\Phi\Phi^T = I_{r-k}$), so $P = \Phi^T\Phi$ is a rank-$(r-k)$ projection with $P_{ii} = p_i \in [0,1]$ and $\sum_i p_i = r - k$. Then:

$$\sum_{j=k+1}^{r} \|Aw_j\|^2 = \sum_{j,i} \alpha_{ji}^2 \sigma_i^2 = \sum_{i=1}^{r} \sigma_i^2 \, p_i.$$

Step 4: elementary rearrangement bound. Since $\sigma_1 \ge \cdots \ge \sigma_r > 0$ and noting that $\sum_{i=1}^{k} p_i = (r-k) - \sum_{i=k+1}^{r} p_i = \sum_{i=k+1}^{r} (1 - p_i)$:

$$\begin{aligned}\sum_{i=1}^{r} \sigma_i^2 p_i - \sum_{i=k+1}^{r} \sigma_i^2 &= \sum_{i=1}^{k} \sigma_i^2 p_i - \sum_{i=k+1}^{r} \sigma_i^2 (1 - p_i) \\ &\ge \sigma_{k+1}^2 \sum_{i=1}^{k} p_i - \sigma_{k+1}^2 \sum_{i=k+1}^{r} (1 - p_i) = 0.\end{aligned}$$

(The inequality uses $\sigma_i \ge \sigma_{k+1}$ for $i \le k$ and $\sigma_i \le \sigma_{k+1}$ for $i \ge k+1$.) Therefore $\|A - B\|_F^2 \ge \sum_{i=k+1}^{r} \sigma_i^2 = \|A - A_k\|_F^2$. $\square$

Applications

1. Image compression

A greyscale image is a matrix $A \in \mathbb{R}^{m\times n}$ of pixel intensities. Storing all mn entries is expensive. The rank-k approximation A_k requires storing U_k $(m \times k)$, Σ_k (k values), and V_k $(n \times k)$: a total of $k(m+n+1)$ numbers.

Compression ratio: $\dfrac{mn}{k(m+n+1)} \approx \dfrac{\min(m,n)}{2k}$ for square images. Breakeven (no saving) at $k \approx mn/(m+n)$.

Quality: Eckart–Young guarantees the truncation error $\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2$. For natural images, singular values decay rapidly, so even $k \ll \min(m,n)$ captures most of the visual content.

```
import numpy as np
import matplotlib.pyplot as plt

# Synthetic "image": sum of low-rank structures + noise
rng = np.random.default_rng(1)
m, n = 128, 128
# True signal: rank-5
U0 = np.linalg.qr(rng.standard_normal((m, 5)))[0]
V0 = np.linalg.qr(rng.standard_normal((n, 5)))[0]
s0 = np.array([100., 60., 30., 15., 5.])
A_true = U0 @ np.diag(s0) @ V0.T
A_noisy = A_true + 3 * rng.standard_normal((m, n))

U, s, Vt = np.linalg.svd(A_noisy, full_matrices=False)

fig, axes = plt.subplots(1, 4, figsize=(14, 3))
axes[0].imshow(A_true, cmap='gray'); axes[0].set_title('True (rank 5)')
for ax, k in zip(axes[1:], [2, 5, 20]):
    Ak = U[:, :k] @ np.diag(s[:k]) @ Vt[:k, :]
    ax.imshow(Ak, cmap='gray')
    err = np.linalg.norm(A_noisy - Ak, 'fro')
    storage_pct = 100 * k * (m + n + 1) / (m * n)
    ax.set_title(f'k={k}, {storage_pct:.1f}% storage\nerr={err:.1f}')
plt.tight_layout(); plt.show()

# Singular value decay
plt.figure(figsize=(5, 3))
plt.semilogy(s[:30], 'o-')
plt.axvline(5, color='r', linestyle='--', label='true rank')
plt.xlabel('index $i$'); plt.ylabel('$\\sigma_i$')
plt.title('Singular value decay'); plt.legend(); plt.show()
```

2. Principal Component Analysis (PCA)

Setup. Given a data matrix $X \in \mathbb{R}^{N\times d}$ (N samples, d features), first *centre*: subtract the column means so each feature has mean zero. The (scaled) sample covariance is

$$C = \tfrac{1}{N} X^T X \in \mathbb{R}^{d\times d},$$

which is PSD. By the spectral theorem $C = V\Lambda V^T$ with V orthogonal and $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$, $\lambda_1 \geq \cdots \geq 0$.

Connection to SVD. The SVD of $X/\sqrt{N} = U\Sigma V^T$ gives $C = V\Sigma^2 V^T$, so the right singular vectors v_i of X *are* the eigenvectors of C. The i-th *principal component* is v_i (direction of i-th most variance). The variance along v_i is σ_i^2/N.

Dimensionality reduction. Project the data onto the top k components:

$$\tilde{X} = XV_k \in \mathbb{R}^{N\times k}, \quad V_k = [v_1 \mid \cdots \mid v_k].$$

Fraction of variance retained: $\sum_{i=1}^k \sigma_i^2 / \sum_i \sigma_i^2$. Eckart–Young guarantees $\tilde{X}$ is the best rank-k summary of X.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(2)

# 2D data: elongated cloud at 45 degrees
N = 300
t = rng.standard_normal(N)
X_raw = np.column_stack([3*t + 0.3*rng.standard_normal(N),
                        3*t + 0.3*rng.standard_normal(N)])
X = X_raw - X_raw.mean(axis=0) # centre

# SVD of X
U, s, Vt = np.linalg.svd(X, full_matrices=False)
print("Singular values:", s.round(2))
print("Variance explained by PC1:", (s[0]**2 / np.sum(s**2) * 100).round(1), "%")

# First principal component = first row of Vt
pc1 = Vt[0]
print("PC1 direction:", pc1.round(3)) # approx [1/sqrt(2), 1/sqrt(2)]

# Projected (1D) data
X_proj = (X @ pc1[:, None]) * pc1[None, :] # rank-1 reconstruction

plt.figure(figsize=(5, 4))
plt.scatter(X[:, 0], X[:, 1], alpha=0.3, s=10, label='data')
plt.plot(*X_proj.T, 'r.', markersize=3, label='projection onto PC1')
plt.quiver(0, 0, *pc1*6, angles='xy', scale_units='xy', scale=1,
          color='k', width=0.02, label='PC1')
plt.axis('equal'); plt.legend(); plt.title('PCA on elongated data'); plt.show()
```

Further applications (lab ideas)

- **Denoising.** Model $M = A_{\text{signal}} + E$ (low-rank signal plus noise). Compute the SVD of M and look for an "elbow" in the singular value plot to pick the truncation rank k.
- **LoRA (Low-Rank Adaptation).** In large language models, weight updates $\Delta W = AB^T$ are constrained to rank $r \ll \min(m, n)$, reducing trainable parameters from mn to $r(m+n)$. This is Eckart–Young applied to gradient updates.

Python: SVD basics and error formula

```
import numpy as np

rng = np.random.default_rng(0)

# ---- Verify A = U Sigma V^T ----
A = rng.standard_normal((4, 6))
U, s, Vt = np.linalg.svd(A, full_matrices=True)
Sigma = np.zeros_like(A, dtype=float)
np.fill_diagonal(Sigma, s)
print("A = U Sigma V^T:", np.allclose(A, U @ Sigma @ Vt))
print("U orthogonal:", np.allclose(U.T @ U, np.eye(U.shape[0])))
print("V orthogonal:", np.allclose(Vt @ Vt.T, np.eye(Vt.shape[0])))

# ---- Eckart-Young error formula ----
B = rng.standard_normal((5, 3))
M = B @ rng.standard_normal((3, 8)) + 0.2 * rng.standard_normal((5, 8))
U2, s2, Vt2 = np.linalg.svd(M, full_matrices=False)
print("\nSingular values:", s2.round(3))
for k in [1, 2, 3, 4]:
    Mk = U2[:, :k] @ np.diag(s2[:k]) @ Vt2[:k, :]
    err_frob = np.linalg.norm(M - Mk, 'fro')
    err_formula = np.sqrt(np.sum(s2[k:]**2))
    print(f"k={k}: ||A-Ak||_F={err_frob:.4f} formula={err_formula:.4f} match={np.isclose(
        err_frob, err_formula)}")
```

16 Determinants (Geometric Approach)

Definition 16.1 (Multilinear alternating form)**.** For $A = [a_1 \mid \cdots \mid a_n]$ with columns $a_i \in \mathbb{R}^n$, the *determinant* is the unique function $\det : \mathbb{R}^{n\times n} \to \mathbb{R}$ such that:

1. det is *multilinear* in the columns.
2. det is *alternating*: swapping two columns flips the sign; repeated columns give 0.
3. $\det(I) = 1$.

Remark 16.1 (Geometric meaning)**.** $|\det(A)|$ is the *volume-scaling factor* of the linear map $x \mapsto Ax$. The sign records orientation. For $n = 2$: $|\det(A)|$ is the area of the parallelogram spanned by the columns.

Proposition 16.1. $\det(A) = 0$ *iff the columns of* A *are linearly dependent.*

Proposition 16.2 (Multiplicativity)**.** $\det(AB) = \det(A)\det(B)$.

Proof. Both sides are multilinear alternating in the columns of B and agree when $B = I$, so they are equal by uniqueness. □

Remark 16.2 (Leibniz formula and cofactor expansion)**.** The Leibniz formula:

$$\det(A) = \sum_{\pi\in S_n} \operatorname{sgn}(\pi) \prod_{i=1}^{n} a_{i,\pi(i)}.$$

Cofactor expansion along row i:

$$\det(A) = \sum_{j=1}^{n} (-1)^{i+j} a_{ij} \det(M_{ij}),$$

where M_{ij} is the $(n-1) \times (n-1)$ minor obtained by deleting row i and column j.

Python example

```
import numpy as np

A = np.array([[2, 1],
              [1, 3]], float)
print("det(A) =", np.linalg.det(A)) # = 5

# 2D area scaling
x = np.array([1.0, 0.0])
y = np.array([0.0, 1.0])
area_before = abs(np.linalg.det(np.column_stack([x, y])))
area_after = abs(np.linalg.det(A @ np.column_stack([x, y])))
print("area before:", area_before, " after:", area_after)
# area_after / area_before == |det(A)| == 5
```

The symmetric group S_n and the sign of a permutation

A *permutation* of $\{1, \ldots, n\}$ is a bijection $\pi : \{1, \ldots, n\} \to \{1, \ldots, n\}$. The set of all $n!$ such bijections, together with composition as the group operation, is the *symmetric group* S_n.

Definition 16.2 (Transposition)**.** A *transposition* is a permutation that swaps exactly two elements and fixes all others. Every permutation can be written as a (non-unique) product of transpositions.

Definition 16.3 (Sign / signature)**.** Although the decomposition into transpositions is not unique, the *parity* of the number of transpositions is an invariant of π. The *sign* (or *signature*) of π is

$$\operatorname{sgn}(\pi) \;=\; \begin{cases} +1 & \text{if } \pi \text{ is a product of an } \textit{even} \text{ number of transpositions,} \\ -1 & \text{if } \pi \text{ is a product of an } \textit{odd} \text{ number of transpositions.} \end{cases}$$

Equivalently, $\operatorname{sgn}(\pi) = \prod_{i<j} \frac{\pi(j)-\pi(i)}{j-i} \in \{+1, -1\}$.

Remark 16.3 (Sign is a group homomorphism)**.** $\operatorname{sgn} : S_n \to \{+1, -1\}$ is a group homomorphism: $\operatorname{sgn}(\pi \circ \sigma) = \operatorname{sgn}(\pi)\operatorname{sgn}(\sigma)$. The *even* permutations (sign $+1$) form the *alternating group* $A_n \trianglelefteq S_n$ of index 2 and order $n!/2$.

Example 16.1 (S_3)**.** S_3 has 6 elements. The identity e and the two 3-cycles $(123), (132)$ are even (sign $+1$); the three transpositions $(12), (13), (23)$ are odd (sign -1).

Uniqueness: the three axioms pin down det

The definition stated three axioms. We now show they are enough to determine det completely — and derive the Leibniz formula as a consequence.

Proposition 16.3 (Uniqueness and Leibniz formula)**.** *The three axioms (multilinearity, alternating, normalisation* $\det(I) = 1$*) determine* det *uniquely. Every function satisfying them equals*

$$\det(A) = \sum_{\pi \in S_n} \operatorname{sgn}(\pi) \prod_{i=1}^{n} a_{i,\pi(i)},$$

where S_n *is the set of all permutations of* $\{1, \ldots, n\}$ *and* $\operatorname{sgn}(\pi) \in \{+1, -1\}$ *is the* signature *of* π *(*$+1$ *if* π *is a product of an even number of transpositions,* -1 *otherwise).*

Proof. Write each column in the standard basis: $a_j = \sum_{i=1}^{n} a_{ij} e_i$. Expand $\det(a_1, \ldots, a_n)$ by multilinearity in every column simultaneously:

$$\det(A) = \sum_{i_1=1}^{n} \cdots \sum_{i_n=1}^{n} a_{i_1 1}\, a_{i_2 2} \cdots a_{i_n n}\ \det(e_{i_1}, e_{i_2}, \ldots, e_{i_n}).$$

There are n^n terms. By the alternating axiom, $\det(e_{i_1}, \ldots, e_{i_n}) = 0$ whenever any two indices coincide. Only the $n!$ terms in which $(i_1, \ldots, i_n)$ is a permutation π of $\{1, \ldots, n\}$ survive.

For any permutation π, decompose it into transpositions. Each transposition swaps two columns and flips the sign once, so

$$\det(e_{\pi(1)}, \ldots, e_{\pi(n)}) = \operatorname{sgn}(\pi)\ \det(e_1, \ldots, e_n) = \operatorname{sgn}(\pi) \cdot 1.$$

Collecting all surviving terms:

$$\det(A) = \sum_{\pi \in S_n} \operatorname{sgn}(\pi) \prod_{i=1}^{n} a_{i,\pi(i)}.$$

This formula is forced by the axioms; there is no freedom left. Hence det is unique. □

Example 16.2 ($n = 2$)**.** $S_2 = \{\mathrm{id}, (12)\}$ with signatures $+1, -1$:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = (+1) \cdot a \cdot d + (-1) \cdot b \cdot c = ad - bc.$$

Computing det via column reduction

The same column operations from §1.6 (which preserved $\operatorname{im} A$) have a precise and simple effect on det.

Proposition 16.4 (Effect of column operations on det)**.**

1. ***Add a multiple:*** $\bar{a}_j \leftarrow \bar{a}_j + \lambda \bar{a}_i$, $i \neq j$*:* det *is* unchanged.
2. ***Scale a column:*** $\bar{a}_j \leftarrow \lambda \bar{a}_j$*:* det *is* multiplied by λ.
3. ***Swap:*** $\bar{a}_i \leftrightarrow \bar{a}_j$*:* det changes sign.

Proof. (1) By multilinearity in column j:

$$\det(\ldots,\, a_j + \lambda a_i,\, \ldots) = \det(\ldots, a_j, \ldots) + \lambda \det(\ldots, a_i, \ldots, a_i, \ldots).$$

The second term has column i repeated, so it vanishes by the alternating axiom.

(2) Multilinearity in column j: $\det(\ldots, \lambda a_j, \ldots) = \lambda \det(\ldots, a_j, \ldots)$.

(3) Direct consequence of the alternating axiom. □

Remark 16.4 (Algorithm for det)**.** Column-reduce A to upper-triangular T using the operations above. Track all scalings $\lambda_1, \ldots, \lambda_s$ and the number of swaps k. Then

$$\det(A) = (-1)^k \cdot \frac{1}{\lambda_1 \cdots \lambda_s} \cdot \prod_{i=1}^{n} t_{ii}.$$

A zero pivot appears iff the columns are linearly dependent, giving $\det(A) = 0$. This is what `np.linalg.det` computes internally (via LU factorisation with partial pivoting).

Proposition 16.5. $\det(A^T) = \det(A)$.

Proof. In the Leibniz formula, $(A^T)_{i,\pi(i)} = a_{\pi(i),i}$. Substitute $\sigma = \pi^{-1}$; as π ranges over S_n, so does σ, and $\operatorname{sgn}(\pi^{-1}) = \operatorname{sgn}(\pi)$. Setting $j = \pi(i)$ (equivalently $i = \sigma(j)$):

$$\prod_{i=1}^{n} a_{\pi(i),i} = \prod_{j=1}^{n} a_{j,\sigma(j)}.$$

Hence $\det(A^T) = \sum_\sigma \operatorname{sgn}(\sigma) \prod_j a_{j,\sigma(j)} = \det(A)$. □

Remark 16.5. Transposing interchanges rows and columns, so $\det(A^T) = \det(A)$ means the alternating property holds equally in the rows. All results stated for columns have row analogues.

Proposition 16.6 ($|\det A|$ = product of singular values)**.** *For $A \in \mathbb{R}^{n\times n}$ with singular values $\sigma_1 \geq \cdots \geq \sigma_n \geq 0$:*

$$|\det(A)| = \sigma_1 \sigma_2 \cdots \sigma_n.$$

Proof. Write $A = U\Sigma V^T$ (SVD, Thm. 15.1). By multiplicativity of det:

$$\det(A) = \det(U) \cdot \det(\Sigma) \cdot \det(V^T).$$

Since $U^T U = I$, taking det gives $\det(U)^2 = 1$, so $\det(U) \in \{+1, -1\}$. Likewise $\det(V^T) \in \{+1, -1\}$. Thus

$$|\det(A)| = |\det(\Sigma)| = \prod_{i=1}^{n} \sigma_i.$$

□

Corollary 16.1. $\det(A) = 0$ *iff A is singular iff some $\sigma_i = 0$ iff* $\operatorname{rank}(A) < n$. *This connects the determinant criterion for invertibility to the kernel/image criterion of §1.6 and the spectral criterion of §4.2.*

Python: determinants in practice

```
import numpy as np
from itertools import permutations

def leibniz_det(A):
    """Compute det(A) via the Leibniz formula (slow, illustrative)."""
    n = A.shape[0]
    total = 0.0
    for pi in permutations(range(n)):
        inv = sum(1 for i in range(n) for j in range(i+1, n) if pi[i] > pi[j])
```

```
        sign = (-1) ** inv
        prod = 1.0
        for i in range(n):
            prod *= A[i, pi[i]]
        total += sign * prod
    return total

A = np.array([[2., 1., 0.],
              [1., 3., 1.],
              [0., 1., 2.]])

print("Leibniz: ", round(leibniz_det(A), 10)) # 8.0
print("np.linalg: ", round(np.linalg.det(A), 10)) # 8.0

# |det| = product of singular values
s = np.linalg.svd(A, compute_uv=False)
print("prod sigma_i:", round(np.prod(s), 10)) # 8.0

# Column operations tracking det
B = A.copy()
B[:, 0] *= 2 # scale col 0 by 2: det(B) = 2 * det(A) = 16
print("After scaling col 0:", round(np.linalg.det(B), 10)) # 16.0

C = A[:, [1, 0, 2]] # swap columns 0 and 1: det changes sign
print("After swap col 0<->1:", round(np.linalg.det(C), 10)) # -8.0
```

17 Tensor Products

Motivation: bilinear maps

Linear maps take one vector input and are linear in it. Many natural operations in the course take *two* vector inputs and are linear in each:

- The inner product $\langle u, v\rangle : V \times V \to \mathbb{R}$ (bilinear, symmetric — §3).
- Matrix multiplication $(A, B) \mapsto AB : \mathbb{R}^{m\times n} \times \mathbb{R}^{n\times p} \to \mathbb{R}^{m\times p}$ (bilinear).
- The outer product $(u, v) \mapsto uv^T : \mathbb{R}^m \times \mathbb{R}^n \to \mathbb{R}^{m\times n}$ (bilinear; appears in the SVD outer-product expansion).

Definition 17.1 (Bilinear map)**.** A map $T : V \times W \to Z$ is *bilinear* if it is linear in each argument with the other held fixed:

$$T(\lambda u + \mu v,\, w) = \lambda T(u, w) + \mu T(v, w), \quad T(v,\, \lambda u + \mu w) = \lambda T(v, u) + \mu T(v, w).$$

More generally, $T : V_1 \times \cdots \times V_k \to Z$ is *k-multilinear* if it is linear in each argument separately.

Remark 17.1. A bilinear map is *not* a linear map from $V \times W$: linearity would require $T(u + v, w+x) = T(u, w)+T(u, x)+T(v, w)+T(v, x)$, but the bilinear condition only gives $T(u+v, w) = T(u, w) + T(v, w)$ (one slot at a time). The difference is the mixed/crossed terms.

The tensor product $V \otimes W$

The tensor product packages the data of all bilinear maps into a single linear-algebra object.

Definition 17.2 (Tensor product of vector spaces)**.** Let $V = \mathbb{R}^m$ with standard basis $\{e_1, \ldots, e_m\}$ and $W = \mathbb{R}^n$ with standard basis $\{f_1, \ldots, f_n\}$. The *tensor product* $V \otimes W$ is the vector space of dimension mn with basis

$$\big\{e_i \otimes f_j : 1 \le i \le m,\ 1 \le j \le n\big\}.$$

For $v = \sum_i v_i e_i \in V$ and $w = \sum_j w_j f_j \in W$, define

$$v \otimes w \;=\; \sum_{i,j} v_i w_j \,(e_i \otimes f_j) \;\in\; V \otimes W.$$

Remark 17.2 (Tensor product = outer product, concretely)**.** The coefficient array of $v \otimes w$ is $(v_i w_j)$, which is exactly the *outer product matrix* $vw^T \in \mathbb{R}^{m\times n}$. So $V \otimes W \cong \mathbb{R}^{m\times n}$ as a vector space, with simple tensors corresponding to rank-1 matrices.

Remark 17.3 (Not every tensor is a simple tensor)**.** The element $e_1 \otimes f_1 + e_2 \otimes f_2 \in V \otimes W$ corresponds to the matrix $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, which has rank 2 and cannot be written as a single vw^T. The general element of $V \otimes W$ is $\sum_{i,j} t_{ij}(e_i \otimes f_j)$, corresponding to an arbitrary matrix $T \in \mathbb{R}^{m\times n}$.

The *tensor rank* of T is the minimum number of simple tensors needed; this equals the matrix rank. The SVD outer-product expansion $A = \sum_{k=1}^{r} \sigma_k u_k v_k^T$ is a decomposition of A into r simple tensors — the minimum possible by Eckart–Young.

Proposition 17.1 (Universal property)**.** *Every bilinear map $T : V \times W \to Z$ factors uniquely through $V \otimes W$: there is a unique linear map $\tilde{T} : V \otimes W \to Z$ such that $T(v, w) = \tilde{T}(v \otimes w)$ for all v, w.*

Conversely, specifying a linear map on $V \otimes W$ is equivalent to specifying a bilinear map on $V \times W$.

Proof sketch. Define $\tilde{T}$ on basis elements by $\tilde{T}(e_i \otimes f_j) = T(e_i, f_j)$ and extend by linearity. This is well-defined and unique. Bilinearity of T ensures $\tilde{T}(v \otimes w) = T(v, w)$ for all simple tensors, hence for all $V \times W$. □

Remark 17.4. The universal property says: *bilinear is the same as linear once you pass through* $\otimes$. It is the same move made in §1.5: every linear map $V \to W$ is determined by what it does to a basis. Here: every bilinear map $V \times W \to Z$ is determined by what it does to basis pairs (e_i, f_j), which becomes a linear map on $V \otimes W$.

The Kronecker product

If $A : V \to V'$ and $B : W \to W'$ are linear maps, the map $(v, w) \mapsto (Av) \otimes (Bw)$ is bilinear from $V \times W$ into $V' \otimes W'$. By the universal property it factors through a linear map $A \otimes B : V \otimes W \to V' \otimes W'$.

Definition 17.3 (Kronecker product)**.** For $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{p\times q}$, the *Kronecker product* $A \otimes B \in \mathbb{R}^{mp\times nq}$ is the block matrix

$$A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{pmatrix}.$$

It acts on simple tensors by $(A \otimes B)(v \otimes w) = (Av) \otimes (Bw)$.

Proposition 17.2 (Mixed-product property)**.** $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$, *whenever the dimensions are compatible.*

Proof. On simple tensors: $(A \otimes B)(C \otimes D)(v \otimes w) = (A \otimes B)((Cv) \otimes (Dw)) = (ACv) \otimes (BDw) = (AC \otimes BD)(v \otimes w)$. Since simple tensors span, equality holds everywhere. □

Corollary 17.1 (Eigenvalues of $A \otimes B$)**.** *If $Av = \lambda v$ and $Bw = \mu w$, then $(A \otimes B)(v \otimes w) = \lambda\mu\,(v \otimes w)$. In particular, if $A \in \mathbb{R}^{m \times m}$ has eigenvalues $\lambda_1, \ldots, \lambda_m$ and $B \in \mathbb{R}^{n \times n}$ has eigenvalues $\mu_1, \ldots, \mu_n$, then $A \otimes B$ has eigenvalues $\{\lambda_i \mu_j\}$.*

Proof. Immediate from the mixed-product property applied to $v \otimes w$. □

Example 17.1 (Solving $AX + XB^T = C$)**.** The *Sylvester equation* $AX + XB^T = C$ arises in control theory and numerical linear algebra. Applying vec (stack columns of X) and using $(I_n \otimes A)\,\mathrm{vec}(X) = \mathrm{vec}(AX)$ and $(B \otimes I_m)\,\mathrm{vec}(X) = \mathrm{vec}(XB^T)$:

$$(I_n \otimes A + B \otimes I_m)\ \mathrm{vec}(X) = \mathrm{vec}(C).$$

This is a standard $mn \times mn$ linear system. It has a unique solution iff A and $-B$ share no eigenvalue — which, by the eigenvalue formula above, means $\lambda_i + \mu_j \neq 0$ for all i, j.

Python: Kronecker products

```python
import numpy as np

A = np.array([[1., 2.],
              [3., 4.]])
B = np.array([[0., 5.],
              [6., 7.]])

K = np.kron(A, B)
print("A kron B:\n", K)
print("shape:", K.shape) # (4, 4)

# Simple tensor = rank-1 matrix = outer product
v = np.array([1., 2., 3.])
w = np.array([4., 5.])
T = np.outer(v, w) # v tensor w
print("\nv outer w:\n", T)
print("rank:", np.linalg.matrix_rank(T)) # 1

# Mixed-product: (A kron B)(u kron w) == (Au) kron (Bw)
A2 = np.array([[1., 0.], [0., -1.]])
B2 = np.array([[2., 1.], [0., 2.]])
u = np.array([1., 2.])
w2 = np.array([3., 4.])
lhs = np.kron(A2, B2) @ np.kron(u, w2)
rhs = np.kron(A2 @ u, B2 @ w2)
print("\nMixed-product check:", np.allclose(lhs, rhs)) # True

# Eigenvalues of A kron B = products of eigenvalues
```

```
lam_A = np.linalg.eigvals(A)
lam_B = np.linalg.eigvals(B)
expected = np.sort([a * b for a in lam_A for b in lam_B]).real
actual = np.sort(np.linalg.eigvals(K)).real
print("Kronecker eigenvalues match:", np.allclose(expected, actual))
```

18 Alternating Forms and the Exterior Algebra

From bilinear to alternating

The determinant is the canonical example of an n-linear *alternating* form. This section develops the full family of alternating forms, of which det is the top member.

Definition 18.1 (k-linear alternating form)**.** A map $\omega : (\mathbb{R}^n)^k \to \mathbb{R}$ is a *k-linear alternating form* (or *k-form*) if:

1. It is linear in each of the k arguments.
2. $\omega(\ldots, v_i, \ldots, v_j, \ldots) = -\omega(\ldots, v_j, \ldots, v_i, \ldots)$ whenever two arguments are swapped.

Equivalently, $\omega(\ldots, v, \ldots, v, \ldots) = 0$ whenever two arguments coincide. Write $\mathrm{Alt}^k(\mathbb{R}^n)$ for the space of all k-forms on $\mathbb{R}^n$.

Remark 18.1 (Alternating $\Leftrightarrow$ sign of S_k acts)**.** The alternating condition is equivalent to saying ω transforms under the sign representation of the symmetric group S_k: for any permutation $\pi \in S_k$,

$$\omega(v_{\pi(1)}, \ldots, v_{\pi(k)}) = \mathrm{sgn}(\pi)\, \omega(v_1, \ldots, v_k).$$

Each transposition contributes one sign flip, and since sgn is a homomorphism the total sign is $\mathrm{sgn}(\pi)$ regardless of how π is factored. This is precisely why the Leibniz formula for det sums over S_n with weights $\mathrm{sgn}(\pi)$.

Example 18.1.

- $\mathrm{Alt}^0(\mathbb{R}^n) = \mathbb{R}$ (constants).
- $\mathrm{Alt}^1(\mathbb{R}^n)$: linear functionals $\mathbb{R}^n \to \mathbb{R}$ (the dual space).
- $\mathrm{Alt}^2(\mathbb{R}^2)$: every 2-form satisfies $\omega(u, v) = c(u_1 v_2 - u_2 v_1)$ for some $c \in \mathbb{R}$. For $c = 1$ this is the signed area of the parallelogram spanned by u and v.
- $\mathrm{Alt}^n(\mathbb{R}^n)$: 1-dimensional, spanned by det (proved in the uniqueness theorem above).
- $\mathrm{Alt}^k(\mathbb{R}^n)$ for $k > n$: must be $\{0\}$, since any $k > n$ vectors in $\mathbb{R}^n$ are linearly dependent and the alternating property forces $\omega = 0$.

Proposition 18.1 ($\dim \mathrm{Alt}^k(\mathbb{R}^n) = \binom{n}{k}$)**.** *A basis for $\mathrm{Alt}^k(\mathbb{R}^n)$ is indexed by k-element subsets $I = \{i_1 < \cdots < i_k\} \subset \{1, \ldots, n\}$: define ω_I by*

$$\omega_I(v_1, \ldots, v_k) = \det\big([v_1 \mid \cdots \mid v_k]_I\big),$$

where $[\cdot]_I$ is the $k \times k$ submatrix using rows I. These $\binom{n}{k}$ forms are linearly independent and span $\mathrm{Alt}^k(\mathbb{R}^n)$.

Proof sketch. Each ω_I is k-linear and alternating (inherited from det). For any k-form ω, its values on all k-tuples are determined by $\omega(e_{i_1}, \ldots, e_{i_k})$ for $i_1 < \cdots < i_k$ (by multilinearity and alternating, exactly as in the Leibniz proof). There are $\binom{n}{k}$ such values, giving dim $\leq \binom{n}{k}$. Linear independence of the ω_I gives equality. □

Remark 18.2 (The dimension table)**.**

k	0	1	2	$\cdots$	$n-1$	n
dim	1	n	$\binom{n}{2}$	$\cdots$	n	1

The table is symmetric: $\binom{n}{k} = \binom{n}{n-k}$. The top entry ($k = n$, dimension 1) is why det is unique up to normalisation.

The wedge product

Definition 18.2 (Wedge product of 1-forms)**.** For linear functionals $\alpha, \beta \in (\mathbb{R}^n)^*$ (1-forms), their *wedge product* $\alpha \wedge \beta$ is the 2-form

$$(\alpha \wedge \beta)(u, v) = \alpha(u)\beta(v) - \alpha(v)\beta(u).$$

More generally, for $\omega \in \mathrm{Alt}^j$ and $\eta \in \mathrm{Alt}^k$:

$$(\omega \wedge \eta)(v_1, \ldots, v_{j+k}) = \frac{1}{j!\,k!} \sum_{\pi \in S_{j+k}} \mathrm{sgn}(\pi)\, \omega(v_{\pi(1)}, \ldots, v_{\pi(j)})\, \eta(v_{\pi(j+1)}, \ldots, v_{\pi(j+k)}).$$

Example 18.2 (Basis 2-forms via wedge)**.** Let e_i^* denote the i-th coordinate functional: $e_i^*(v) = v_i$. Then

$$(e_i^* \wedge e_j^*)(u, v) = u_i v_j - u_j v_i = \det \begin{pmatrix} u_i & v_i \\ u_j & v_j \end{pmatrix} = \omega_{\{i,j\}}(u, v).$$

So $\{e_i^* \wedge e_j^* : i < j\}$ is exactly the basis from Prop. 18.1.

Proposition 18.2 (Properties of $\wedge$)**.**

1. ***Associativity:*** $(\omega \wedge \eta) \wedge \zeta = \omega \wedge (\eta \wedge \zeta)$.
2. ***Anti-commutativity:*** $\omega \wedge \eta = (-1)^{jk}\, \eta \wedge \omega$ *for* $\omega \in \mathrm{Alt}^j$, $\eta \in \mathrm{Alt}^k$.
3. ***Top form is the determinant:***

$$e_1^* \wedge e_2^* \wedge \cdots \wedge e_n^* = \det.$$

Proof of (3). By the definition of $\wedge$ applied $n-1$ times: $(e_1^* \wedge \cdots \wedge e_n^*)(v_1, \ldots, v_n) = \sum_\pi \mathrm{sgn}(\pi) \prod_i e_i^*(v_{\pi(i)}) = \sum_\pi \mathrm{sgn}(\pi) \prod_i (v_{\pi(i)})_i$. Re-indexing with $\sigma = \pi^{-1}$: $= \sum_\sigma \mathrm{sgn}(\sigma) \prod_i (v_i)_{\sigma(i)} = \det([v_1 \mid \cdots \mid v_n])$. □

The determinant as a map on alternating forms

A linear map $A : \mathbb{R}^n \to \mathbb{R}^n$ acts on k-forms by *pullback*: for $\omega \in \mathrm{Alt}^k(\mathbb{R}^n)$,

$$(A^*\omega)(v_1, \ldots, v_k) = \omega(Av_1, \ldots, Av_k).$$

This is again a k-form, so A^* maps $\mathrm{Alt}^k \to \mathrm{Alt}^k$.

For $k = n$: since $\mathrm{Alt}^n(\mathbb{R}^n)$ is 1-dimensional (spanned by det), the map A^* must be multiplication by some scalar $c(A)$:

$$(A^* \det)(v_1, \ldots, v_n) = \det(Av_1, \ldots, Av_n) = c(A)\, \det(v_1, \ldots, v_n).$$

Evaluating at $(e_1, \ldots, e_n)$ gives $c(A) = \det(A)$.

Theorem 18.1 ($\det(AB) = \det(A)\det(B)$ via pullback)**.** *Pullback is functorial:* $(AB)^* = B^* \circ A^*$. *Therefore*

$$\det(AB)\cdot\det = (AB)^*\det = B^*(A^*\det) = B^*(\det(A)\cdot\det) = \det(A)\cdot B^*\det = \det(A)\det(B)\cdot\det\,.$$

Evaluating at $(e_1, \ldots, e_n)$*:* $\det(AB) = \det(A)\det(B)$.

Remark 18.3. This proof works because Alt^n is 1-dimensional: the multiplicativity of det is a consequence of there being *no room* for any other behaviour. The Leibniz-formula proof of multiplicativity used the same idea implicitly; the pullback argument makes the structure explicit.

Python: alternating forms and volume

```python
import numpy as np

def wedge_2form(i, j, u, v):
    """(e_i* wedge e_j*)(u, v) = u[i]*v[j] - u[j]*v[i]"""
    return u[i]*v[j] - u[j]*v[i]

# In R^3: the three basis 2-forms
u = np.array([1., 2., 3.])
v = np.array([4., 5., 6.])
w12 = wedge_2form(0, 1, u, v) # = 1*5 - 4*2 = -3
w13 = wedge_2form(0, 2, u, v) # = 1*6 - 4*3 = -6
w23 = wedge_2form(1, 2, u, v) # = 2*6 - 5*3 = -3
print("omega_{01}(u,v) =", w12)
print("omega_{02}(u,v) =", w13)
print("omega_{12}(u,v) =", w23)

# These are the components of -(u x v)
print("u x v =", np.cross(u, v)) # [-3, 6, -3]

# Top form: det([u,v,w]) = (e0*^e1*^e2*)(u,v,w)
w = np.array([7., 8., 10.])
A = np.column_stack([u, v, w])
print("\ndet([u,v,w]) =", round(np.linalg.det(A), 10))

# Pullback: det(Av_1,...,Av_n) = det(A) * det(v_1,...,v_n)
rng = np.random.default_rng(0)
M = rng.standard_normal((4, 4))
V = rng.standard_normal((4, 4))
lhs = np.linalg.det(M @ V)
rhs = np.linalg.det(M) * np.linalg.det(V)
print("det(MV) = det(M)*det(V):", np.isclose(lhs, rhs)) # True
```

Exercise 18.1.

1. Show that $\omega(u, v) = u_1v_2 - u_2v_1$ on $\mathbb{R}^2$ satisfies $\omega(Au, Av) = \det(A)\,\omega(u, v)$ for every $A \in \mathbb{R}^{2\times 2}$. Conclude: *area scales by* $|\det(A)|$.

2. In $\mathbb{R}^3$, the three components of $u \times v$ are $(u_2v_3 - u_3v_2,\ u_3v_1 - u_1v_3,\ u_1v_2 - u_2v_1)$. Express each as a basis 2-form $\omega_I(u, v)$. Conclude that $u \times v$ encodes all of $\mathrm{Alt}^2(\mathbb{R}^3)$.